

la-video-engine

Análisis Técnico Detallado de `la-video-engine`

Este informe contiene una descripción técnica exhaustiva de la arquitectura, APIs, protocolos y lógica de almacenamiento de `la-video-engine`, el motor principal de grabación, streaming y persistencia multimedia en el ecosistema NVR de Lanaccess.

1. Resumen Ejecutivo y Propósito del Servicio

Definición y Rol

`la-video-engine` es el componente de backend de alto rendimiento del NVR (Network Video Recorder). Sus responsabilidades principales consisten en:

- **Captura y Grabación**: Ingestar flujos de vídeo digital (H.264/H.265) desde cámaras IP a través de pipelines multimedia y persistirlos en disco bajo un formato propietario de alta fiabilidad (`fs2`).
- **Streaming y Reproducción**: Transmitir flujos de vídeo en directo y grabaciones históricas tanto por streaming RTSP tradicional (mediante `gst-rtsp-server`) como por segmentación HTTP Live Streaming (HLS) directamente a navegadores web (como la aplicación frontend en Angular 19).
- **Mantenimiento y Telemetría**: Gestionar la detección de discos SATA, realizar análisis de salud física de los soportes de almacenamiento (S.M.A.R.T.) y monitorizar las colas y la latencia del sistema.
- **Servidor de Almacenamiento Persistente de Base de Datos**: Proveen un servidor de E/S de bloques a bajo nivel que permite a SQLite operar de forma transaccional directamente sobre particiones de disco en crudo, garantizando inmunidad a cortes de corriente bruscos.

Pila Tecnológica

- **Lenguaje**: C++17/C++20 compilado con CMake.
- **Framework Multimedia**: GStreamer (para la creación, enrutamiento y control de pipelines de vídeo/audio).
- **Servidor Web**: `libmicrohttpd` (MHD), un servidor HTTP embebido rápido y ligero con soporte para Epoll.
- **IPC (Comunicación entre Procesos)**: Systemd D-Bus (`sd-bus`) que implementa interfaces tanto de cliente como de servidor.
- **Base de Datos**: SQLite, encapsulado mediante un servidor socket UNIX personalizado que lee/escribe en bruto sectores del disco.
- **Monitoreo de Hardware**: `libudev` para la detección en tiempo real de unidades de disco en los puertos SATA.

Interacción con el Hardware y Sistema Operativo

1. **Detección de Discos**: Utiliza `libudev` para monitorizar eventos de inserción/extracción en los 4 puertos SATA físicos (`MAX\_NUM\_DISKS = 4`) expuestos a través de la controladora PCIe SATA del hardware SoC (identificada por el expansor PCIe `33800000.pcie`).
2. **Optimizaciones de Memoria**: A nivel de sistema operativo, el servicio interactúa con el subsistema de memoria configurando Huge Pages de 2048 KB

(`/sys/kernel/mm/hugepages/hugepages-2048kB/nr\_hugepages`) al arrancar el programa para reducir el overhead en la traducción de memoria virtual a física durante la copia de búferes de vídeo a gran velocidad.

3. ****Descriptor de Archivos****: Eleva los límites del sistema en descriptores de ficheros abiertos simultáneamente (`RLIMIT\_NOFILE`) fijando tanto el límite blando como el duro en `4096`.

4. ****Mapeo Físico de Disco****: Realiza escrituras alineadas de bloques y sectores omitiendo la capa de sistema de archivos tradicional de Linux mediante llamadas a bajo nivel de su propio port (`fs\_port`).

5. ****Diagnóstico S.M.A.R.T.****: Ejecuta el comando de sistema `smartctl -a --json /dev/sdX` mediante tuberías POSIX (`popen`) para recopilar la telemetría S.M.A.R.T. de los discos conectados.

2. Arquitectura Interna y Patrones de Diseño

Estructura de Directorios y Patrón Arquitectónico

El repositorio sigue un diseño modular enfocado en la separación de responsabilidades y en la concurrencia:

- `src/main.cpp`: Punto de entrada del programa. Gestiona el ciclo de vida del servicio: inicializa Huge Pages, `fs\_port`, descriptores, GStreamer, el gestor de peticiones asíncronas, el `pipeline\_manager`, la caché de búsquedas de grabaciones, y finalmente arranca los servidores socket UNIX y HTTP (`libmicrohttpd`).
- `src/core/`: Núcleo de control de vídeo.
 - `include/video\_engine.hpp` y `pipeline\_manager.hpp`: Modelan el contexto de la aplicación y la gestión de pipelines multimedia.
 - `source/video\_engine\_server.cpp`: Implementa el servidor D-Bus bajo la interfaz `com.lanaccess.videoengine`.
 - `source/client/la-core-legacy/`: Implementa un cliente D-Bus que interactúa con el servicio heredado `com.lanaccess.properties` para actualizar alarmas del sistema (`send\_system\_alarm`) y reiniciar el equipo si hay un error en disco (`wr\_reboot\_for\_disk\_error`).
- `src/appnet/web/`: Capa del servidor web.
 - `source/web\_server\_init.cpp`: Registro de endpoints y arranque del demonio `libmicrohttpd` con un pool de hilos (`MHD\_OPTION\_THREAD\_POOL\_SIZE = 2`) y límites de conexión por IP.
 - `source/cgi/`: Controladores de peticiones (CGI handlers) para HLS, segmentos, estadísticas del motor y telemetría de discos.
- `src/appnet/db/`: Servidor de bloques para la base de datos de persistencia local.
 - `source/db\_server\_core.cpp`: Bucle de escucha en el socket UNIX `/tmp/io.sock`. Actúa como puente traduciendo operaciones de E/S de archivos SQLite a lecturas/escrituras en crudo a nivel de sector en la partición reservada para bases de datos (`FILE\_MANAGER\_PARTITION\_FOR\_DB`).
- `src/disks\_information/`: Capa de monitoreo de almacenamiento con `libudev` que implementa un bucle de monitorización en un hilo secundario (`monitor\_loop`).
- `src/fs\_port/`: Abstracción y control del almacenamiento físico propietario (inicialización física de discos, soporte RAID, escritura y lectura de bloques estructurados `fs2`).

```

```mermaid
graph TD
 subgraph Web_App_Client [Web App / Client]
 Angular["web-core-view (Angular 19)"]
 end

 subgraph la-video-engine [la-video-engine]
 MHD["libmicrohttpd Web Server (Puerto 8080)"]
 PM["Pipeline Manager (GStreamer)"]
 DB_Srv["DB Block Server (UNIX Socket: /tmp/io.sock)"]
 FS_Port["fs_port (Capa HD / FS2 / RAID)"]
 Disks["Disks Information (libudev)"]
 Searcher["Searcher & Query Cache"]
 DBus_Srv["D-Bus Server Interface"]
 end

 subgraph System_Bus_OS [System Bus & OS]
 DBus_Prop["com.lanaccess.properties (D-Bus)"]
 OS_Disk["Raw Partitions /dev/sdX"]
 Udev["udev daemon"]
 end

 Angular -->|GET /hls-play.m3u8| MHD
 Angular -->|GET /segment.ts| MHD
 Angular -->|GET /disks/smart| MHD
 MHD -->|Itera tokens| Searcher
 MHD -->|Lee bloques de vídeo| FS_Port
 FS_Port -->|Lecturas/Escrituras alineadas| OS_Disk
 Disks -->|Monitoriza eventos SATA| Udev
 DB_Srv -->|Traduce peticiones E/S| FS_Port
 DBus_Srv <-->|com.lanaccess.videoengine| DBus_Prop
```

```

Gestión de Concurrencia y Estado

- **Concurrencia HTTP**: `libmicrohttpd` utiliza el modo `MHD_USE_EPOLL_INTERNAL_THREAD` con un tamaño de pool de hilos de 2 y timeouts de conexión configurados a 120 segundos.
- **Asincronía y Executors**: Para la interacción D-Bus y tareas no bloqueantes, el servicio emplea ejecutores dedicados (`m_executor`, `m_reboot_executor` en `LaCoreLegacy_Client`) que despachan lambdas de C++ de forma asíncrona sobre hilos dedicados para evitar bloquear la ejecución de streaming o grabación en tiempo real.
- **Evitar la fragmentación (Arena Allocator)**: Para gestionar el tráfico masivo de fragmentos de vídeo de forma concurrente sin sobrecargar el asignador de memoria (`malloc`), la clase `pipeline_manager` cuenta con un asignador de búferes dedicado (`buffer_arena`). Este aloja bloques alineados fijos de gran tamaño (por ejemplo, `Buffer8MBPlayer`) reutilizándolos constantemente mediante punteros alineados a sectores del disco (múltiplos de 512 bytes).

- **Protección de datos (Locks)**: Utiliza mutexes de tipo wrapper (`TrackedMutex`) para realizar seguimientos de adquisición y evitar condiciones de carrera al acceder en caliente al estado de los discos conectados y a los caches de los grabadores.

3. Catálogo de Endpoints y APIs (REST / HTTP)

Todos los endpoints HTTP son peticiones **GET** procesadas mediante callbacks de tipo `CgiHandler` y gestionadas por `web_server_cgi_dispatch`.

`/version`

- **Verbo**: GET
- **Propósito**: Devuelve la versión actual del firmware de vídeo.
- **Headers especiales**: `Content-Type: application/json`
- **Payload de Respuesta Exitoso (200 OK)**:

```
```json
{version: "1.0.4"}
```
```

`/stats.json`

- **Verbo**: GET
- **Propósito**: Expone estadísticas detalladas de rendimiento en caliente del motor de vídeo.
- **Headers especiales**: `Content-Type: application/json`
- **Payload de Respuesta Exitoso (200 OK)**:

```
```json
{
 "pipelines": {
 "recorders": 8,
 "receivers": 8,
 "rtsp_servers": 1
 },
 "buffer_arena": {
 "allocated_blocks": 4,
 "free_blocks": 12,
 "block_size_bytes": 8388608
 },
 "latency_metrics_ms": {
 "avg_write_latency": 1.42,
 "max_write_latency": 5.12
 }
}
```
```

`/disks/smart`

- **Verbo**: GET

- **Propósito**: Proporciona el estado de salud y diagnóstico S.M.A.R.T. de todos los discos SATA detectados físicamente.
- **Lógica interna**: Recorre los puertos SATA, obtiene el nombre del dispositivo de disco (ej. `/dev/sda`) a través del servicio de detección por `udev`, y ejecuta el comando asíncrono `smartctl -a --json <ruta_disco>`.
- **Headers especiales**: `Content-Type: application/json`
- **Payload de Respuesta Exitoso (200 OK)**:

```
```json
[
 {
 "sata_port": 0,
 "smart_data": {
 "device": {
 "name": "/dev/sda",
 "info_name": "/dev/sda [SAT]"
 },
 "smart_status": {
 "passed": true
 },
 "temperature": {
 "current": 37
 }
 }
 }
]
```
```

`/hls-play.m3u8`

- **Verbo**: GET
- **Propósito**: Genera la lista de reproducción `.m3u8` dinámicamente para la reproducción en formato HLS de un intervalo de grabaciones históricas.
- **Parámetros de consulta (Query params)**:
 - `token` (String, Obligatorio): Token único de la sesión de reproducción.
 - `start` (Integer, Opcional): Timestamp Unix (segundos) de inicio del intervalo (def: 0).
 - `end` (Integer, Opcional): Timestamp Unix (segundos) de fin del intervalo (def: 0).
 - `variant` (String, Opcional): Si no se pasa, el endpoint devuelve una lista maestra de tipo variante que redirige al propio endpoint añadiendo `variant=1`.
- **Respuestas Especiales**:
 - **Redirección de Variante**: Si no se incluye el parámetro `variant`, devuelve la especificación de ancho de banda y códec HLS apuntando a la variante real:

```
```ini
#EXTM3U
#EXT-X-VERSION:3
```

```
#EXT-X-STREAM-INF:BANDWIDTH=4000000,RESOLUTION=1920x1080,CODECS="hvc1.1.6.L93.B0"
```

```
hls-play.m3u8?token=abc123xyz&start=1718016000&end=1718020000&variant=1
```
```

(Retornado con Content-Type: `application/vnd.apple.mpegurl; charset=utf-8`).

- ****VOD Playlist (Con `variant=1`)**:**

```
``ini
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-PLAYLIST-TYPE:VOD
#EXT-X-TARGETDURATION:11
#EXT-X-DISCONTINUITY
#EXT-X-PROGRAM-DATE-TIME:2026-06-10T12:00:00.000000
#EXTINF:10.035,
segment.ts?id=1024&buffer=0&codec=2&master_id=1024
#EXTINF:9.982,
segment.ts?id=1024&buffer=1&codec=2&master_id=1024
#EXT-X-ENDLIST
```
```

- **\*\*Manejo de Errores\*\*:**

- Si falta el parámetro `token`, devuelve **\*\*400 Bad Request\*\*** con un payload JSON de error: `{"status": 400, "message": "'token' missing."}`.
- Si el token no se encuentra en la caché de búsqueda (`searcher`), o si el intervalo solicitado no intersecta con ninguna grabación persistente, genera una lista HLS vacía y limpia para que el player de Angular no falle:

```
``ini
#EXTM3U
#EXT-X-VERSION:3
#EXT-X-PLAYLIST-TYPE:VOD
#EXT-X-TARGETDURATION:10
#EXT-X-ENDLIST
```
```

`/segment.ts`

- ****Verbo**:** GET

- ****Propósito**:** Sirve el fragmento de vídeo correspondiente en formato MPEG-2 Transport Stream (MPEG-TS) traduciendo en caliente el búfer binario crudo del sistema de archivos `fs2`.

- ****Parámetros de consulta (Query params)**:**

- `id` (Integer, Obligatorio): Identificador de la grabación en disco.
- `buffer` (Integer, Obligatorio): Índice del bloque de datos binario dentro del archivo.
- `codec` (Integer, Opcional): `1` para H.264, `2` para H.265 (def: 1).
- `master\_id` (Integer, Opcional): ID de la grabación de origen/maestra para recuperar cabeceras si el códec es H.265.

- ****Headers especiales**:** `Content-Type: video/MP2T`

- ****Manejo de Errores**:**

- ****400 Bad Request**:** Parámetros corruptos, no numéricos o incompletos (`Invalid Params`).
- ****404 Not Found**:** Si la grabación solicitada no existe en el grabador (`Recording not found`), o si el búfer está vacío (`Empty Segment`).
- ****500 Internal Server Error**:** Si la lectura de disco subyacente a través de la API del sistema de archivos falla (`Read Failed`).

4. Protocolos en Tiempo Real (WebSockets / SSE)

Arquitectura de Notificación en Tiempo Real

`la-video-engine` **no** expone conexiones WebSockets o flujos SSE directamente al exterior. El diseño de la arquitectura distribuye esta tarea de la siguiente manera:

1. **Canal Interno (D-Bus)**: `la-video-engine` emite señales D-Bus en el bus del sistema ante eventos de grabación, cambios en el estado de los discos SATA monitoreados y fallas/alarmas de hardware.
2. **Pasarela de Notificación (API Gateway/Notifier)**: Otro servicio web ligero del ecosistema NVR (por ejemplo, el gateway web o el servicio `la-core-notifier`) escucha las señales D-Bus de `la-video-engine` y actúa como puente de WebSockets hacia el cliente frontend en Angular (`web-core-view`).
3. **Subprotocolos Comunes**:
 - `la-core-notifier`: Transmite notificaciones en caliente como `IO\_NOTIFY` (cambios en el estado de relés de entrada y salida) y `ERRORS\_NOTIFY` (fallos críticos de lectura/escritura).
 - `la-core-alarm`: Canaliza eventos de seguridad física y alarmas del grabador (`DB\_ENGINE\_ALARM\_NOTIFY`) que el frontend Angular captura para renderizar advertencias visuales en la interfaz web de control.

5. Mecanismo de Configuración, Bloqueo y Esquemas

La gestión administrativa de configuraciones, políticas de persistencia, generación de esquemas de validación y control de concurrencia de escrituras se realiza de manera centralizada en el sistema.

Interfaz de Configuración D-Bus

El servicio `la-video-engine` actúa estrictamente como **consumidor** de configuraciones. En lugar de procesar peticiones HTTP `PATCH` directamente, consulta y sincroniza su estado comunicándose con el servicio central de propiedades `com.lanaccess.properties` a través de D-Bus:

- **Lectura de Propiedades**: Implementada mediante `dbus\_querier\_call\_get\_property`. Por ejemplo, para obtener el nombre del hardware o la versión de firmware llama al método `Get` de la interfaz D-Bus:
 - Servicio de Destino: `com.lanaccess.properties`
 - Ruta de Objeto: `/com/lanaccess/properties`
 - Parámetros: Token de propiedad (ej. `ro.hardware.name`, `config.device.users`, `config.io.inputs`).
- **Escritura de Propiedades**: Realizada a través de `dbus\_querier\_call\_set\_property`, invocando el método `Set` en el bus.

Ciclo de Bloqueo Pesimista (Pessimistic Locking)

- ****Rol del Backend Central****: Las operaciones de adquisición (``acquireLock``), renovación periódica y liberación (``releaseLock``) del token de sesión de edición de configuración las gestiona el servicio ``com.lanaccess.properties`` en el bus D-Bus.
- ****Validación****: Este servicio centralizado valida que cualquier petición de actualización (``PATCH`` o ``PUT``) adjunte un ``token`` activo y coincida con el ``seed`` temporal generado al abrir la sesión en la aplicación Angular. ``la-video-engine`` no tiene visibilidad de este ciclo de bloqueo; únicamente recibe notificaciones de que la configuración ha sido consolidada.

Fusión de Cambios (Patch Merging) e Impacto en Hardware

- Cuando el usuario guarda cambios en la configuración (ej. alterar el bitrate de una cámara o el modo de grabación), el backend central realiza la fusión parcial de propiedades y detecta si los nodos alterados requieren un reinicio completo del servicio o de hardware.
- Si el cambio requiere reiniciar módulos físicos o pipelines de GStreamer, el backend de configuración expone en la cabecera HTTP la bandera ``X-Reboot-Required``.
- En caso de errores graves de hardware en caliente detectados por ``la-video-engine`` (ej. degradación irrecuperable en un sector de escritura de disco en modo RAID), el propio motor solicita de forma asíncrona al core del sistema un reinicio de emergencia llamando a ``wr_reboot_for_disk_error`` sobre el bus D-Bus.

6. Pipeline de Vídeo, Almacenamiento y Telemetría

Gestión de Flujos de Vídeo (GStreamer Pipelines)

``la-video-engine`` construye pipelines dinámicas en GStreamer para dos funciones críticas:

1. ****Ingesta (Recorders / Receivers)****: Recibe el streaming H.264/H.265 desde las cámaras IP. Si la cámara sufre desconexión, el motor entra en el estado ``NoSignal`` o ``Stopped`` (según su máquina de estados). Al restaurarse, conmuta dinámicamente y vuelve a encolar búferes en disco (``NewBuffer`` / ``NewFrame``).
2. ****Distribución (RTSP Transmitters)****: Utiliza ``gst-rtsp-server`` para levantar servidores de transmisión de vídeo bajo demanda (RTSP). Soporta:
 - RTSP sobre TCP/UDP (Unicast y Multicast).
 - RTSP seguro con cifrado TLS (mediante la función ``pipeline_manager_create_rtsp_server_with_tls``).
 - Sincronización horaria absoluta (RTP-Info transmite marcas de tiempo NTP absolutas basadas en el año 1900 para que los reproductores puedan sincronizar múltiples cámaras frame a frame).

Estructura de Almacenamiento Físico (``fs2`` y RAID)

El almacenamiento implementado evita la fragmentación utilizando una estructura de grabación directa por sectores a disco en una partición propietaria.

A nivel de hardware, el servicio implementa lógica personalizada para coordinar escrituras paralelas bajo diferentes modos de RAID físico (definidos en ``hd_defines.h``):

- ****RAID_MODE_DISABLED (99)****: Graba en discos individuales sin redundancia ni combinación.

- ****RAID_MODE_0_BIG (0)****: Striping lineal básico. Concatena hasta 4 discos físicos para formar un único volumen lógico continuo de almacenamiento de grabaciones, optimizando el rendimiento agregando ancho de banda.
- ****RAID_MODE_1_MIRROR (1)****: Mirroring. Escribe simultáneamente en dos discos en paralelo. Si un disco falla, conmuta al superviviente y actualiza el estado a través de D-Bus (``io_hd_input_value_set``) para alertar al sistema de que requiere una reconstrucción (``rebuilding``).
- ****RAID_MODE_DOUBLE_MIRROR (2)****: Doble Mirroring. Distribuye las escrituras en pares de discos redundantes (espejo redundante de 4 unidades).
- ****RAID_MODE_DINAMICO (3) / N+1****: Esquema dinámico de recuperación y paridad.

Servidor de Bloques de Base de Datos para Persistencia

Dado que el NVR debe ser resistente a pérdidas repentinas de alimentación eléctrica sin usar sistemas de ficheros pesados sobre las particiones de datos, ``la-video-engine`` implementa un servidor que aloja un socket UNIX en ``/tmp/io.sock``.

Cuando SQLite inicia, interactúa con este socket enviando estructuras empaquetadas ``IoRequest`` que contienen operaciones de bloques. El motor de vídeo mapea directamente estas peticiones a offsets fijos dentro de la partición cruda dedicada:

- ****Espacio de Metadatos****: Comienza en el offset ``0`` del sector, con un tamaño de 4 KB. Almacena la estructura ``FsDBSuperBlock`` que incluye la firma mágica ``0xAABBCCDD`` y el tamaño lógico actual de las bases de datos.
- ****SQLite Journal / WAL****:
 - ``main.db-journal`` (Slot 0): Mapeado físicamente en el offset ``4 KB`` con una capacidad máxima de 64 MB.
 - ``main.db-wal`` (Slot 1): Mapeado físicamente en el offset ``64 MB + 4 KB`` con una capacidad de 64 MB (Write-Ahead Log).
- ****Base de Datos de Grabaciones e Índices****:
 - ``main.db`` (Slot 2): Mapeada físicamente a partir de ``128 MB + 4 KB`` con una capacidad fija máxima de 800 MB.

Las lecturas y escrituras sectorizadas se ejecutan invocando:

- ``fs2_partition_raw_read_sectors(...)``
- ``fs2_partition_raw_write_sectors(...)``

Cualquier escritura que no esté alineada exactamente a sectores de 512 bytes pasa por un proceso interno de lectura-modificación-escritura (RMW - Read-Modify-Write) en el búfer de la página de 4096 bytes para proteger la integridad de los datos.

7. Integración con el Frontend (web-core-view)

El frontend en Angular 19 (``web-core-view``) interactúa con el ecosistema NVR de forma coordinada. A continuación se detallan las secuencias de comunicación para las operaciones principales.

Flujo de Reproducción Histórica HLS (Vídeo bajo Demanda)

Este diagrama describe cómo la aplicación de Angular solicita autorización y visualiza grabaciones históricas de vídeo utilizando los endpoints HTTP expuestos por `la-video-engine`.

```
```mermaid
sequenceDiagram
 autonumber
 actor Angular as web-core-view (Angular 19)
 participant Gateway as API Gateway (NVR Portal)
 participant Engine as la-video-engine (HTTP Server)
 participant DB as Almacenamiento fs2 (Discos)

 Angular->>Gateway: POST /api/playback/request-session { cam_id, r_start, r_end }
 Note over Gateway: Genera token de reproducción y registra sesión en caché global
 Gateway-->>Angular: 200 OK { token }

 Angular->>Engine: GET
 /hls-play.m3u8?token=xyz123&start=1718016000&end=1718020000
 Note over Engine: Valida token. Busca en la caché de grabaciones históricas de los discos
 Engine-->>Angular: 200 OK (Redirección a variante con variant=1)

 Angular->>Engine: GET
 /hls-play.m3u8?token=xyz123&start=1718016000&end=1718020000&variant=1
 Note over Engine: Abre ficheros indexados en fs2. Genera playlist con referencias a segment.ts
 Engine-->>Angular: 200 OK (Fichero de Playlist .m3u8 indexado)

 loop Descarga de Segmentos HLS
 Angular->>Engine: GET /segment.ts?id=1024&buffer=0&codec=2&master_id=1024
 Note over Engine: Reserva búfer de 8MB del Buffer Arena.
Lee bloques de sectores directamente del disco.
 Engine->>DB: fs2_partition_raw_read_sectors(sector_start, sector_count)
 DB-->>Engine: Bloque de datos binario (H.265)
 Note over Engine: Procesa y reempaqueta NALs (AVCC a Annex B) e inserta VPS/SPS/PPS.
Muxea la pista en un flujo MPEG-TS (188 bytes/paquete).
 Engine-->>Angular: 200 OK (Flujo binario video/MP2T)
 Note over Engine: Libera búfer al Buffer Arena
 end
```
```

Flujo de Diagnóstico de Discos (S.M.A.R.T.)

El frontend en Angular solicita la información física de salud de las unidades SATA para mostrarla visualmente en el panel de administración técnica.

```
```mermaid
sequenceDiagram
 autonumber
```

actor Angular as web-core-view (Angular 19)  
participant Gateway as API Gateway (NVR Portal)  
participant Engine as la-video-engine (HTTP Server)  
participant OS as Sistema Operativo / smartctl

Angular->>Gateway: GET /api/system/storage/diagnostics  
Gateway->>Engine: GET /disks/smart  
Note over Engine: Itera SATA 0..3.<br/>Obtiene rutas de disco físicas via udev.  
loop Por cada puerto SATA detectado  
    Engine->>OS: popen("smartctl -a --json /dev/sdX")  
    OS-->>Engine: Raw JSON del diagnóstico del disco  
    Note over Engine: Limpia caracteres adicionales y encapsula por SATA port  
end  
Engine-->>Gateway: 200 OK (JSON formateado en array)  
Gateway-->>Angular: 200 OK (Retorna datos de salud de discos)

...

### ### Flujo de Configuración y Reboot por Degradación de Hardware

En este flujo, si se realiza un cambio administrativo o se detecta un error de E/S grave en un disco RAID, se activa el mecanismo de reinicio para reconfigurar el hardware.

```mermaid

sequenceDiagram

 autonumber

 actor Angular as web-core-view (Angular 19)
 participant Gateway as API Gateway (NVR Portal)
 participant Prop as com.lanaccess.properties (D-Bus)
 participant Engine as la-video-engine
 participant Legacy as com.lanaccess.core-legacy (D-Bus)

 Note over Angular, Prop: Configuración de Parámetros Administrativos

 Angular->>Gateway: POST /api/config/set { token, properties }

 Gateway->>Prop: Invoca SetList(properties) con validación de token de bloqueo

 Prop-->>Gateway: OK

 Gateway-->>Angular: 200 OK [Headers: X-Reboot-Required]

 Note over Engine, Legacy: Reacción ante Fallos Críticos de Disco (Almacenamiento)

 Engine->>Engine: Detecta fallo de escritura masivo en sector RAID

 Engine->>Prop: Actualiza estado D-Bus (io_hd_input_value_set)

 Engine->>Legacy: Invoca D-Bus wr_reboot_for_disk_error(disk_index)

 Note over Legacy: Planifica y fuerza reinicio seguro del equipo NVR

...

la-db-engine

Informe Técnico de Arquitectura: Servicio `la-db-engine`

Este informe técnico proporciona un análisis profundo del repositorio del servicio `la-db-engine`, que actúa como el motor de almacenamiento de base de datos dentro del ecosistema del NVR (Network Video Recorder). Este análisis está estructurado para servir como material de referencia y soporte en la redacción de tu Trabajo de Fin de Grado (TFG).

1. Resumen Ejecutivo y Propósito del Servicio

Definición y Rol

El servicio `la-db-engine` es el motor de base de datos relacional del ecosistema NVR. Su responsabilidad principal es centralizar, persistir y gestionar los datos transaccionales de alarmas generados por el sistema de videovigilancia.

Actúa como un `servidor de base de datos desacoplado`, exponiendo un canal de comunicación entre procesos (IPC) mediante `gRPC` para que otros servicios del backend (como `la-core-alarm` y `la-core-notifier`) puedan registrar y consultar eventos sin necesidad de conectarse directamente a un archivo de base de datos local o preocuparse por concurrencias complejas de bajo nivel.

Pila Tecnológica

- * `Lenguaje de programación:` C++ (Estándar C++23).
- * `Interfaces de comunicación:`
 - * `gRPC:` Utilizado como interfaz de control interna y de alta velocidad, escuchando en un socket de dominio Unix (`unix:///tmp/la-db-engine/la_db_engine.sock`).
 - * `Libmicrohttpd:` Servidor HTTP embebido liviano para exponer métricas de rendimiento y verificación del estado del servicio.
- * `Motor de Base de Datos:` `SQLite3` embebido.
- * `Gestor de compilación:` CMake.

Interacción con el Hardware y el Sistema Operativo

A diferencia de las aplicaciones tradicionales donde SQLite escribe directamente en el sistema de archivos del disco local mediante llamadas estándar al kernel (`open`, `write`, `read`), `la-db-engine` implementa un `mecanismo de abstracción de entrada/salida a nivel de hardware/sistema operativo`.

El servicio registra un driver virtual de base de datos personalizado a través del framework SQLite VFS (Virtual File System) denominado `remote_vfs`.

- * Cuando SQLite realiza operaciones lógicas de disco (por ejemplo, leer una página o vaciar el diario WAL), estas llamadas son interceptadas por `remote_vfs`.
- * El VFS redirige las solicitudes empaquetadas en una estructura binaria a través de un socket UNIX (`/tmp/io.sock`) a un proceso gestor de E/S centralizado (I/O manager).
- * Esto previene que el motor de base de datos requiera acceso físico directo a los discos de almacenamiento de vídeo, permitiendo una arquitectura de almacenamiento centralizada y segura.

2. Arquitectura Interna y Patrones de Diseño

Patrón Arquitectónico y Estructura de Directorios

El código del repositorio está organizado de forma modular y sigue principios de diseño orientados a servicios:

```
...
├── CMakeLists.txt          # Configuración de compilación CMake
├── CHANGELOG.md            # Historial de versiones del servicio
├── src
│   ├── proto
│   │   └── database.proto  # Definición del contrato gRPC (Servicio y Mensajes)
│   ├── version.hpp        # Define la versión y el nombre del servicio
│   └── server
│       ├── main.cpp        # Punto de entrada de la aplicación
│       ├── core
│       │   ├── database_manager.h/.cpp # Singleton gestor de SQLite3
│       │   ├── remote_vfs.h/.cpp      # Driver virtual personalizado de SQLite
│       │   └── io_protocol_v1.h       # Definición del protocolo binario de E/S
│       ├── grpc
│       │   └── database_service_impl.h/.cpp # Implementación del servidor gRPC
│       └── web
│           ├── include      # Cabeceras del servidor Web Libmicrohttpd
│           └── source       # Implementaciones HTTP y controladores CGI
...

```

* **Patrón Singleton (Instancia Única):** La clase `DatabaseManager` se implementa como un Singleton para garantizar que exista una sola conexión abierta y coordinada con SQLite.

* **Patrón Proxy / Adapter (SQLite VFS):** `remote_vfs` actúa como un adaptador que traduce llamadas del motor SQLite a mensajes binarios del protocolo `IoRequest` / `IoResponse`.

* **Patrón Client-Server (gRPC):** El servicio gRPC centraliza la manipulación de datos en hilos separados a través del protocolo HTTP/2 (gRPC sobre socket UNIX).

Gestión de Concurrencia y Estado

El servicio maneja la concurrencia en múltiples niveles:

1. Bloqueo a Nivel de Base de Datos (SQLite):

* Se inicializa SQLite con la opción `SQLITE_OPEN_NOMUTEX`, delegando el control de exclusión mutua de hilos a la capa de C++.

* Se aplican directivas de rendimiento críticas mediante PRAGMAs durante el inicio:

```
```sql
```

```
PRAGMA locking_mode=EXCLUSIVE; -- Bloqueo exclusivo del archivo de BD para evitar overhead de locks de archivos del SO.
```

```
PRAGMA journal_mode=WAL; -- Write-Ahead Logging para permitir lecturas concurrentes sin bloquear escrituras.
```

PRAGMA synchronous=NORMAL; -- Reduce las sincronizaciones de disco agresivas, equilibrando seguridad ante fallos y velocidad.

## 2. **Sincronización en C++ (`std::shared\_mutex`):**

- \* Para evitar colisiones entre hilos que llaman al servidor gRPC concurrentemente, `DatabaseManager` contiene un `std::shared\_mutex db\_mutex`.

- \* **Lecturas (`query`):** Emplean un bloqueo compartido (`std::shared\_lock lock(db\_mutex)`), lo que permite que múltiples hilos consulten la base de datos de manera paralela.

- \* **Escrituras (`execute`):** Emplean un bloqueo exclusivo (`std::unique\_lock lock(db\_mutex)`), deteniendo a otros lectores y escritores para asegurar la consistencia del archivo de datos.

## 3. **Límite de Almacenamiento (Trigger FIFO):**

- \* Para evitar que la base de datos crezca indefinidamente y agote el almacenamiento, `DatabaseManager` configura un trigger a nivel de base de datos que limita la tabla `ALARMS` a un máximo de **10,000 registros** (`MAX\_ALARM\_ENTRIES`).

- \* El trigger elimina las alarmas más antiguas (las de menor `id`) cada vez que se inserta un nuevo registro:

```
```sql
CREATE TRIGGER limit_alarms_trigger
AFTER INSERT ON ALARMS
BEGIN
    DELETE FROM ALARMS WHERE id NOT IN (
        SELECT id FROM ALARMS ORDER BY id DESC LIMIT 10000
    );
END;
```
```

---

## ## 3. Catálogo de Endpoints y APIs

El servicio expone dos tipos de APIs: un endpoint REST/CGI de telemetría y diagnóstico, y un servicio interno gRPC transaccional.

### ### A. API REST / HTTP (Mapeada mediante CGI)

Escucha en el puerto HTTP **60891** (por defecto) y solo acepta el verbo `GET`.

#### #### 1. `GET /version`

- \* **Propósito:** Retorna la versión del servicio.

- \* **Cabeceras:** `Content-Type: application/json`

- \* **Ejemplo de respuesta (JSON sin comillas en llaves):**

```
```json
{service_name: "la-db-engine", version: "1.0.9"}
```
```

#### #### 2. `GET /status`



\* **Propósito:** Retorna el estado detallado de salud, estadísticas de lectura/escritura física del VFS y detalles de las tablas SQLite.

\* **Cabeceras:** `Content-Type: application/json`

\* **Estructura del Payload de Respuesta:**

```
``json
{
 "status": "ok",
 "service": "la-db-engine",
 "version": "1.0.9",
 "stats": {
 "total_reads": 1258,
 "total_writes": 412
 },
 "sqlite": {
 "vfs": "remote_vfs",
 "journal_mode": "wal",
 "page_size": 4096,
 "page_count": 124,
 "total_size_bytes": 507904,
 "total_size_mb": 0.48
 },
 "tables": [
 {
 "name": "ALARMS",
 "entries": 150
 }
]
}
```

\* **Manejo de Errores HTTP:**

\* Si se llama con métodos diferentes a `GET` (por ejemplo, `POST`), el servidor responde rechazando la conexión (`MHD\_NO`).

\* Si se intenta acceder a una ruta inexistente, devuelve un código **404 Not Found** con el texto plano:

```
...
```

```
db engine API
```

```

```

```
available: /version, /status
```

```
...
```

\* Si falla la lectura de propiedades de SQLite internamente, el campo `"status"` cambia a `"error"` o `"degraded"`, pero se devuelve la estructura JSON con un código HTTP `200 OK` (salvo que falle el proceso HTTP por completo).

```

```

### ### B. API Interna (gRPC)

Escucha en el socket Unix: `unix:///tmp/la-db-engine/la_db_engine.sock`.

#### 1. `rpc Execute(ExecuteRequest) returns (ExecuteResponse)`

\* **Propósito:** Ejecuta sentencias SQL de modificación (INSERT, UPDATE, DELETE o DDL).

\* **Mensaje de Entrada (`ExecuteRequest`):**

\* `string sql`: La consulta SQL a ejecutar.

\* **Mensaje de Respuesta (`ExecuteResponse`):**

\* `bool success`: Indica si la ejecución fue exitosa.

\* `int64 rows\_affected`: Cantidad de filas modificadas.

\* `string error\_message`: Mensaje de error detallado (si aplica).

\* `int64 last\_insert\_id`: El identificador autoincremental de la fila insertada.

#### 2. `rpc Query(QueryRequest) returns (stream QueryResponse)`

\* **Propósito:** Realiza consultas SQL de lectura (`SELECT`) y transmite las filas resultantes una por una mediante streaming de gRPC.

\* **Mensaje de Entrada (`QueryRequest`):**

\* `string sql`: Sentencia SELECT a ejecutar.

\* **Mensaje de Respuesta (`QueryResponse`):**

\* `map<string, string> row`: Un diccionario clave-valor por fila, mapeando el nombre de la columna al valor serializado como cadena de texto. Si un valor es nulo en base de datos, se devuelve como `"NULL"`.

---

#### ## 4. Protocolos en Tiempo Real (WebSockets / SSE)

El servicio `la-db-engine` **no expone conexiones WebSockets o SSE directas** al frontend. Esta es una decisión de diseño para mantener el motor de base de datos con un único propósito (almacenamiento) y evitar la sobrecarga de conexiones de clientes externos.

Sin embargo, juega un rol crucial en el pipeline en tiempo real:

\* Los servicios que gestionan WebSockets (como `la-core-alarm` y `la-core-notifier`) se conectan a `la-db-engine` vía gRPC.

\* Cuando `la-core-alarm` recibe un evento de alarma de hardware por D-Bus u otro protocolo, llama a `la-db-engine::Execute` para persistirla.

\* Una vez guardada de forma segura, el propio servicio de alarmas emite el evento a través de su servidor WebSocket hacia el cliente Angular (`web-core-view`).

---

#### ## 5. Mecanismo de Configuración, Bloqueo y Esquemas

Dado que `la-db-engine` es un motor de base de datos persistente genérico, **no gestiona directamente esquemas de configuración de hardware ni sistemas de bloqueo pesimista**. Esas lógicas corresponden al servicio de configuración central del NVR.

Sin embargo, el servicio sí interactúa con un sistema de bajo nivel mediante su arquitectura VFS:

### ### Protocolo de E/S Virtualizado (Custom Remote VFS)

El archivo `io_protocol_v1.h` define cómo se empaquetan las transacciones físicas de E/S. Cada llamada a archivo SQLite se traduce en una trama de red binaria de longitud fija:

```
```cpp
#pragma pack(push, 1)
struct IoRequest
{
    IoOperation op;           // Identificador de operación (uint8_t)
    int64_t offset;           // Offset de archivo para lectura/escritura
    uint64_t size;            // Tamaño de los datos
    char filename[64];        // Archivo objetivo (ej: "main.db")
};

struct IoResponse
{
    int32_t result;           // Resultado de la operación (bytes leídos/escritos o código de
    // error)
};
#pragma pack(pop)
```
```

### #### Ciclo de Operación de E/S de la Base de Datos:

1. **\*\*Conexión inicial:\*\*** En `remote_open`, se abre un socket UNIX a `/tmp/io.sock`. Se envía un request con la operación `IoOperation::OPEN` y el nombre del archivo.
2. **\*\*Lectura/Escritura de páginas:\*\***
  - \* Para leer, se envía `IoRequest` con `op = IoOperation::READ`, el offset de la página y el tamaño.
  - \* El socket devuelve `IoResponse`. Si `result > 0`, se procede a leer los bytes del buffer inmediatamente a través del flujo del socket.
  - \* Para escribir, se envía el struct `IoRequest` con `op = IoOperation::WRITE` seguido directamente del buffer de bytes de datos. El servidor de I/O responde confirmando los bytes escritos en un `IoResponse`.
3. **\*\*Sincronización:\*\*** Para asegurar la durabilidad del diario de transacciones de SQLite, se envía `IoOperation::SYNC` que fuerza un flush de caché físico en el almacenamiento remoto.

---

### ## 6. Pipeline de Vídeo, Almacenamiento o Telemetría

`la-db-engine` no ingiere flujos de vídeo (no maneja RTSP, WHEP o HLS). Su papel en el almacenamiento de vídeo se limita a:

- \* **\*\*Indexación:\*\*** Guardar referencias temporales, marcas de tiempo e identificadores de cámara en la tabla `ALARMS` para que el frontend pueda vincular una alarma con una grabación específica de vídeo.

\* \*\*Métricas expuestas en `/status`:\*\* Muestra la telemetría del VFS de SQLite (`total\_reads`, `total\_writes`, `page\_count`, etc.), lo que permite monitorear si el almacenamiento remoto está experimentando cuellos de botella de latencia de E/S.

---

## ## 7. Integración con el Frontend (web-core-view)

El frontend en Angular 19 (`web-core-view`) interactúa indirectamente con `la-db-engine`. La base de datos es la fuente de verdad persistente.

A continuación se muestra el diagrama de secuencia que ilustra el flujo de comunicación típico desde que se produce una alerta de hardware hasta su lectura y despliegue en la interfaz web de Angular:

```
sequenceDiagram
 participant HW as Hardware / Sensores
 participant AlarmSrv as Servicio la-core-alarm (Backend)
 participant DbEng as la-db-engine (Backend)
 participant IoSrv as Gestor E/S /tmp/io.sock
 participant Web as Angular 19 (web-core-view)

 Note over HW, Web: Flujo 1: Recepción y Registro de Alarma en Tiempo Real
 HW->>AlarmSrv: D-Bus Event / Señal de E/S Física
 AlarmSrv->>DbEng: gRPC: Execute(sql="INSERT INTO ALARMS (timestamp, source, type, status, is_read, ...) VALUES (...)")
 DbEng->>IoSrv: IPC (Socket UNIX): IoRequest(op=WRITE, filename="main.db", offset, size) + payload bytes
 IoSrv-->>DbEng: IPC: IoResponse(result=size_written)
 DbEng-->>AlarmSrv: gRPC: ExecuteResponse(success=true, last_insert_id=452)
 AlarmSrv-->>Web: WebSocket Broadcast: { event: "ALARM_NOTIFY", id: 452, camera_num: "2", type: "MOTION" }
 Note over Web: El frontend Angular 19 renderiza dinámicamente la alarma en la UI

 Note over HW, Web: Flujo 2: Consulta Histórica de Alarmas desde el Frontend
 Web->>AlarmSrv: GET /api/alarms?read=false (o endpoint equivalente)
 AlarmSrv->>DbEng: gRPC: Query(sql="SELECT * FROM ALARMS WHERE is_read=0 ORDER BY id DESC")
 DbEng->>IoSrv: IPC: IoRequest(op=READ, filename="main.db", offset, size)
 IoSrv-->>DbEng: IPC: IoResponse(result=size) + db_page_bytes
 DbEng-->>AlarmSrv: gRPC Stream: QueryResponse(row={id: "452", type: "MOTION", is_read: "0", ...})
 AlarmSrv-->>Web: HTTP Response: JSON Array [{ id: 452, type: "MOTION", read: false }]
```

Note over HW, Web: Flujo 3: Marcar Alarma como Leída

```
Web->>AlarmSrv: POST /api/alarms/452/read
AlarmSrv->>DbEng: gRPC: Execute(sql="UPDATE ALARMS SET is_read=1 WHERE
id=452")
DbEng->>IoSrv: IPC: IoRequest(op=WRITE, offset, size) + payload bytes
IoSrv-->>DbEng: IPC: IoResponse(result=size_written)
DbEng-->>AlarmSrv: gRPC: ExecuteResponse(success=true, rows_affected=1)
AlarmSrv-->>Web: HTTP 200 OK / { success: true }
...
```

### Detalle de Campos Clave de la Tabla `ALARMS` vinculados al Frontend:

- \* `camera\_num`: Asocia el evento a un canal de vídeo físico en la interfaz de visualización.
- \* `is\_read`: Controla si el icono en la barra de notificaciones del frontend Angular debe parpadear o mostrar un globo de alerta.
- \* `is\_locked`: Evita que una alarma sea eliminada manualmente o rotada por el trigger FIFO de 10,000 registros si el usuario decide protegerla.

la-core-alarm

## # Análisis de Arquitectura y Diseño Técnico del Servicio: la-core-alarm

Este documento contiene un análisis técnico exhaustivo y de bajo nivel del servicio `la-core-alarm`, un componente fundamental en el ecosistema de videovigilancia y grabación (NVR - \*Network Video Recorder\*) de Lanaccess. Su responsabilidad principal es procesar, persistir y notificar eventos de alarma en tiempo real a los clientes conectados (incluyendo la aplicación frontend `web-core-view` desarrollada en Angular 19).

---

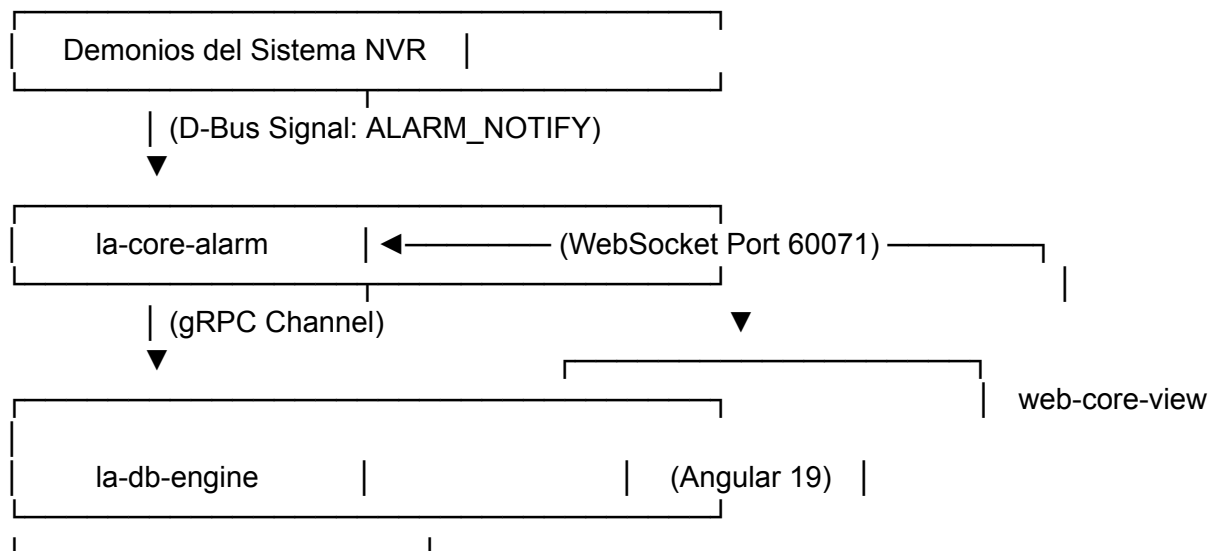
### ## 1. Resumen Ejecutivo y Propósito del Servicio

#### ### Definición y Rol en el Ecosistema NVR

`la-core-alarm` actúa como el **mediador de eventos y logs de alarma** del NVR. Este servicio opera de forma continua en segundo plano (demonio de Linux) realizando tres tareas esenciales:

- Consumo de Señales del Sistema (D-Bus):** Escucha de forma pasiva los eventos de alarma generados por otros demonios del sistema (como fallos de hardware, accesos no autorizados, analíticas de vídeo o eventos físicos de Entrada/Salida).
- Persistencia (gRPC / SQLite):** Se comunica con el motor de base de datos local (`la-db-engine`) para almacenar el historial de alarmas de manera persistente.
- Distribución en Tiempo Real (WebSockets):** Mantiene una pasarela de comunicación bidireccional de baja latencia con el navegador del operador mediante WebSockets, permitiendo la visualización instantánea de alarmas en curso y facilitando la gestión histórica (marcar como leídas, archivar o eliminar).

...



...

#### ### Pila Tecnológica

- **Lenguaje de Programación:** C++23. Utiliza intensivamente características modernas del estándar como hilos cooperativos (`std::jthread`) y tokens de parada (`std::stop_token`).

- **Controlador de Señales y Buses (IPC):** ``libsystemd`` (``sd-bus``) para escuchar e integrarse con el bus de mensajes del sistema (D-Bus).
- **Capa de Base de Datos / RPC:** ``gRPC`` y ``Protocol Buffers`` (Protobuf v3) para comunicarse de forma eficiente mediante Sockets Unix con el demonio de base de datos.
- **Servidor WebSockets:** ``libwebsockets`` (LWS), una biblioteca en C altamente optimizada para la gestión de sockets no bloqueantes mediante bucles de eventos `epoll/poll`.
- **Procesamiento JSON:** ``jsoncpp`` para codificación y decodificación de payloads.

### ### Interacción con el Hardware/SO

El servicio ``la-core-alarm`` no interactúa directamente con drivers a nivel de kernel ni sistemas de archivos de vídeo. En su lugar, delega estas operaciones y se apoya en abstracciones del sistema operativo:

- **D-Bus System/User Bus:** Se suscribe al canal de comunicación de `systemd` en Linux para capturar señales emitidas en la interfaz ``com.lanaccess.sysevents``. Se adapta a entornos de desarrollo y producción evaluando la variable de entorno ``LA_CORE_LEGACY_PC``. Si está establecida como ``"yes"``, se conecta al bus de sesión del usuario (``sd_bus_open_user``); de lo contrario, se enlaza al bus del sistema global (``sd_bus_open_system``).
- **Unix Domain Sockets:** Establece un canal gRPC local conectándose al socket Unix expuesto en ``unix:///tmp/la-db-engine/la_db_engine.sock``. Esto evita el overhead de la pila TCP/IP para la persistencia local.
- **Sockets TCP (Red):** Levanta un puerto TCP visible de red (``60071``) dedicado al protocolo de WebSockets.

---

## ## 2. Arquitectura Interna y Patrones de Diseño

El repositorio está organizado bajo un enfoque modular y orientado a eventos, desacoplando el transporte de entrada (D-Bus), el de salida/gestión (WebSocket) y la lógica de almacenamiento (gRPC).

...

```
src/
├── main.cpp # Punto de entrada y gestión del ciclo de vida
├── client.h / client.cpp # Implementación del cliente gRPC para la DB
├── include/
│ ├── thread_safe_queue.hpp # Cola concurrente genérica (SafeQueue)
│ └── db-client/ # Capa cliente gRPC
│ ├── include/ # APIs y estructuras de datos de alarma
│ └── source/ # Hilos y manejadores (inserción, actualización, etc.)
├── dbus/ # Capa de integración D-Bus
│ ├── include/ # Definición de colas de emisión y configuraciones
│ └── source/ # Oyente de señales y formateador WebSocket
└── websocket/ # Capa de servidor WebSockets (LWS)
 └── include/ # Wrappers RAII para LWS y definiciones de protocolo
```



```
└─ source/
lectura/escritura
...
```

```
Servidor LWS y callbacks no bloqueantes de
```

### ### Flujo y Comunicación entre Módulos (Gestión de Estado y Concurrency)

El demonio utiliza una arquitectura multihilo sustentada por **cinco hilos concurrentes** que se comunican mediante colas de productor-consumidor seguras para hilos (`SafeQueue` basada en `std::mutex` y `std::condition_variable`):

```
```mermaid
```

```
flowchart TD
```

```
    subgraph D-Bus Module
```

```
        DBusListener[dbus_listen_alarms_thread]
```

```
        DBusEmitter[ws_emit_alarms_thread]
```

```
    end
```

```
    subgraph Database Client Module
```

```
        DBClient[db_client_thread]
```

```
        DBCommand[db_client_command_thread]
```

```
        GRPC[DatabaseClient gRPC Stub]
```

```
    end
```

```
    subgraph WebSocket Module
```

```
        LWS[web_socket_server_thread]
```

```
    end
```

```
%% Entrada de alarmas externas
```

```
SystemD[D-Bus System Bus] -->|Signal: ALARM_NOTIFY| DBusListener
```

```
DBusListener -->|Push AlarmData| Q1[global_alarm_queue]
```

```
Q1 -->|Pop| DBClient
```

```
%% Interacción base de datos gRPC (Alarma Start/End)
```

```
DBClient -->|gRPC Execute/Query| GRPC
```

```
GRPC -->|Push EmissionData| Q2[global_emission_queue]
```

```
Q2 -->|Pop| DBusEmitter
```

```
DBusEmitter -->|Enqueue JSON & lws_cancel_service| LWS
```

```
%% Interacción del Cliente WebSocket (web-core-view)
```

```
web-core-view((web-core-view)) <-->|WebSocket Connection| LWS
```

```
LWS -->|Push CommandData| Q3[global_command_queue]
```

```
Q3 -->|Pop| DBCommand
```

```
DBCommand -->|gRPC Execute/Query| GRPC
```

```
DBCommand -->|Enqueue Response JSON| LWS
```

```
```
```

1. `dbus_listen_alarms_thread`: Ejecuta un bucle cerrado sobre `sd_bus_wait` y `sd_bus_process`. Cuando captura un evento `ALARM_NOTIFY`, desempaqueta los 8

parámetros del payload y los encola en la cola global `global\_alarm\_queue` como un struct `AlarmData`.

2. **db\_client\_thread**: Consume de `global\_alarm\_queue`.

- Si la alarma es de inicio (`status = "start"`), realiza un `INSERT` en la tabla `ALARMS` mediante el cliente gRPC y añade el registro retornado (con el `id` asignado por la base de datos) a la cola `global\_emission\_queue`.

- Si la alarma es de fin (`status = "end"`), busca el inicio de alarma correspondiente en la base de datos, calcula la asociación y actualiza la fila estableciendo la fecha de fin (`end\_date`). Posteriormente, encola la alarma finalizada en `global\_emission\_queue`.

3. **ws\_emit\_alarms\_thread**: Monitoriza `global\_emission\_queue`. Extrae los eventos de alarma procesados, los serializa en formato JSON bajo el esquema `DB\_ENGINE\_ALARM\_NOTIFY` y los añade a las colas de salida de todas las conexiones activas en el módulo WebSocket, despertando el bucle de LWS con `lws\_cancel\_service`.

4. **db\_client\_command\_thread**: Monitoriza `global\_command\_queue`. Procesa las órdenes enviadas por los clientes WebSockets (marcar lectura, archivar, borrar u obtener histórico), ejecuta las sentencias SQL pertinentes en la base de datos a través de gRPC, y encola la confirmación correspondiente en el módulo WebSocket.

5. **web\_socket\_server\_thread**: Ejecuta el bucle de eventos `lws\_service` de libwebsockets. Administra la conexión física de los clientes, procesa tramas parciales, convierte tramas JSON entrantes en `CommandData` para el hilo de comandos, y gestiona la escritura asíncrona hacia los clientes conectados.

---

### ## 3. Catálogo de Endpoints y APIs (REST / HTTP)

> [!IMPORTANT]

> **El servicio `la-core-alarm` no implementa ni expone ningún Endpoint REST / HTTP.**

>

> En su lugar, el backend utiliza un modelo de comunicación orientado a mensajes sobre WebSockets en el puerto TCP `60071`. Todas las operaciones CRUD e histórico que habitualmente residirían en endpoints REST se realizan a través del envío de objetos JSON identificados por el campo `"event"`.

A continuación se detalla la equivalencia lógica de este catálogo de operaciones:

| Operación Lógica | Equivalente REST | Evento WebSocket (Request) | Evento WebSocket (Response) |
|------------------|------------------|----------------------------|-----------------------------|
|------------------|------------------|----------------------------|-----------------------------|

|      |      |      |      |
|------|------|------|------|
| :--- | :--- | :--- | :--- |
|------|------|------|------|

|                            |                                         |              |                      |
|----------------------------|-----------------------------------------|--------------|----------------------|
| <b>Consultar Historial</b> | `GET /alarms?select_date=...&quant=...` | `ALARMS_GET` | `ALARMS_GET_SUCCESS` |
|----------------------------|-----------------------------------------|--------------|----------------------|

|                          |                                     |              |                      |
|--------------------------|-------------------------------------|--------------|----------------------|
| <b>Marcar como Leída</b> | `PATCH /alarms/:id {is_read: true}` | `ALARM_READ` | `ALARM_READ_SUCCESS` |
|--------------------------|-------------------------------------|--------------|----------------------|

|                        |                                         |                 |                         |
|------------------------|-----------------------------------------|-----------------|-------------------------|
| <b>Archivar Alarma</b> | `PATCH /alarms/:id {is_archived: true}` | `ALARM_ARCHIVE` | `ALARM_ARCHIVE_SUCCESS` |
|------------------------|-----------------------------------------|-----------------|-------------------------|

|                        |                      |                |                        |
|------------------------|----------------------|----------------|------------------------|
| <b>Eliminar Alarma</b> | `DELETE /alarms/:id` | `ALARM_DELETE` | `ALARM_DELETE_SUCCESS` |
|------------------------|----------------------|----------------|------------------------|

#### ### Estructura General de Errores

Si ocurre una anomalía al procesar una petición en el hilo ``db_client_command_thread`` (por ejemplo, un fallo en la consulta gRPC), el backend captura el fallo en la estructura interna ``ExecuteResult`` / ``QueryResult``, imprimiendo el error en el flujo stderr:

```
```cpp
std::cerr << "[DB-Command Thread] Failed to update alarm " << db_id << ": " <<
result.error_message << std::endl;
```
```

En la versión actual del código, si una mutación de datos falla en base de datos, el backend simplemente descarta la emisión del mensaje de confirmación (``ALARM_*_SUCCESS``), lo que permite al frontend Angular determinar el fallo por *timeout* o por la ausencia de actualización del estado en el stream del evento.

---

#### ## 4. Protocolos en Tiempo Real (WebSockets)

El servidor WebSocket está implementado sobre la biblioteca ``libwebsockets`` y escucha en el puerto `60071`.

#### ### Endpoint de Conexión

```
```
ws://<NVR_IP>:60071/
```

* **Subprotocolo Requerido (Handshake Header):** `la-core-notifier`
```

#### ### Gestión de Conexiones y Máquina de Estados

- **Límite de Concurrencia:** El servidor cuenta con una directiva de seguridad que rechaza nuevas conexiones si el mapa interno supera el máximo de sesiones simultáneas:

```
```cpp
static constexpr unsigned int MAX_CONCURRENT_CONNECTIONS = 10;
```
```

Al alcanzar las 10 conexiones, el callback ``LWS_CALLBACK_ESTABLISHED`` retorna ``-1``, abortando inmediatamente el handshake TCP/WS.

- **Gestión de Desconexiones:** Al interrumpirse la conexión, el callback ``LWS_CALLBACK_CLOSED`` elimina la sesión del mapa ``active_connections`` de manera segura liberando memoria.

- **Escrituras Asíncronas y Fragmentación:** `libwebsockets` utiliza un modelo reactivo no bloqueante. Cuando hay mensajes pendientes de enviar a un cliente, el backend invoca ``lws_callback_on_writable(wsi)``. Al recibir el callback ``LWS_CALLBACK_SERVER_WRITEABLE``, el sistema:

1. Extrae el mensaje de ``pending_messages_to_send_queue``.
2. Reserva memoria incluyendo el padding de cabecera obligatorio de LWS (``LWS_PRE`` bytes de desfase).
3. Ejecuta ``lws_write`` con el flag ``LWS_WRITE_TEXT``.
4. Si ocurre una escritura parcial, guarda la posición en ``bytes_sent_of_current_message`` y vuelve a registrar el socket para escritura en el siguiente ciclo del bucle.

---

### ### Catálogo Detallado de Mensajes (JSON Contracts)

#### ##### A. Eventos Enviados por el Cliente (web-core-view -> Backend)

##### ##### 1. Obtener Histórico de Alarmas (`ALARMS\_GET`)

Utilizado para realizar paginación o carga inicial de alarmas anteriores a una fecha determinada.

- \*\*JSON Payload:\*\*

```
```json
{
  "event": "ALARMS_GET",
  "payload": {
    "select_date": "2026-06-10T12:00:00Z",
    "quant": "50"
  }
}
```
```

\*Nota: `select\_date` actúa como cursor temporal para la paginación y `quant` determina el tamaño de la página.\*

##### ##### 2. Marcar Alarma como Leída (`ALARM\_READ`)

- \*\*JSON Payload:\*\*

```
```json
{
  "event": "ALARM_READ",
  "payload": [
    {
      "db_id": "125",
      "read": true
    }
  ]
}
```
```

##### ##### 3. Archivar Alarma (`ALARM\_ARCHIVE`)

- \*\*JSON Payload:\*\*

```
```json
{
  "event": "ALARM_ARCHIVE",
  "payload": [
    {
      "db_id": "125",
      "archive": true
    }
  ]
}
```
```

...

#### ##### 4. Eliminar Registro de Alarma (`ALARM\_DELETE`)

- \*\*JSON Payload:\*\*

```
```json
{
  "event": "ALARM_DELETE",
  "payload": [
    {
      "db_id": "125"
    }
  ]
}
```
```

---

#### #### B. Eventos Enviados por el Servidor (Backend -> web-core-view)

##### ##### 1. Notificación de Alarma en Tiempo Real (`DB\_ENGINE\_ALARM\_NOTIFY`)

Emitido cada vez que una alarma cambia de estado (se crea o finaliza).

- \*\*JSON Payload:\*\*

```
```json
{
  "event": "DB_ENGINE_ALARM_NOTIFY",
  "payload": [
    {
      "topic": "db/alarm/125",
      "db_id": "125",
      "timestamp": "2026-06-10T12:44:00Z",
      "source": "hw_sensor",
      "status": "start",
      "alarm_id": "9",
      "description": "Detector volumétrico entrada activado",
      "priority": "2",
      "type": "digital_input",
      "camera_num": "4",
      "start_date": "2026-06-10T12:44:00Z",
      "end_date": ""
    }
  ]
}
```
```

##### ##### 2. Confirmación de Lectura (`ALARM\_READ\_SUCCESS`)

- \*\*JSON Payload:\*\*

```
```json
{
```

```
"event": "ALARM_READ_SUCCESS",
"payload": {
  "db_id": "125",
  "read": true
}
}
...
```

3. Confirmación de Archivado (`ALARM\_ARCHIVE\_SUCCESS`)

- **JSON Payload:**

```
```json
{
 "event": "ALARM_ARCHIVE_SUCCESS",
 "payload": {
 "db_id": "125",
 "archive": true
 }
}
...
```

#### ##### 4. Confirmación de Eliminación (`ALARM\_DELETE\_SUCCESS`)

- \*\*JSON Payload:\*\*

```
```json
{
  "event": "ALARM_DELETE_SUCCESS",
  "payload": {
    "db_id": "125"
  }
}
...
```

5. Respuesta a Consulta de Histórico (`ALARMS\_GET\_SUCCESS`)

- **JSON Payload:**

```
```json
{
 "event": "ALARMS_GET_SUCCESS",
 "payload": [
 {
 "db_id": "125",
 "timestamp": "2026-06-10T12:44:00Z",
 "source": "hw_sensor",
 "status": "end",
 "alarm_id": "9",
 "description": "Detector volumétrico entrada activado",
 "priority": "2",
 "type": "digital_input",
 "camera_num": "4",
 "start_date": "2026-06-10T12:44:00Z",

```

```

 "end_date": "2026-06-10T12:45:10Z",
 "is_read": true,
 "is_archived": false
 }
]
}
...

```

---

## ## 5. Mecanismo de Configuración, Bloqueo y Esquemas

> [!NOTE]  
 > **la-core-alarm** no implementa ni participa en el sistema de Configuración, Bloqueo (Pessimistic Locking) ni Esquemas del NVR.

Dentro de la arquitectura global del sistema, los mecanismos de bloqueo (`acquireLock` / `releaseLock``) basados en tokens/seeds, la generación de esquemas de validación JSON (`/properties/schema``) y la combinación de cambios parciales (PATCH merging) para evaluar reinicios de hardware son responsabilidad exclusiva de **otros microservicios especializados** del backend (por ejemplo, el servicio central de configuración).

El servicio `la-core-alarm`` se limita de forma estricta a un modelo operacional de base de datos transaccional lineal y a la distribución de eventos reactivos sobre la tabla `ALARMS``. No almacena ni procesa ficheros de configuración del dispositivo ni implementa la cabecera `X-Reboot-Required``.

---

## ## 6. Pipeline de Vídeo, Almacenamiento o Telemetría

> [!NOTE]  
 > **la-core-alarm** no implementa ingesta ni streaming de vídeo ni gestiona almacenamiento RAID físico.

Toda la lógica relativa al streaming multimedia (WHOP, HLS indexado con ficheros `.m3u8``, o RTSP), así como el formateo de discos y redundancia RAID (disabled, mirror, double-mirror, big, n+1), recae sobre el servicio **la-video-engine`** (también denominado **Mantos-Rec**).

La única relación funcional que tiene `la-core-alarm`` con los flujos de vídeo es el almacenamiento de metadatos de las alarmas (específicamente la asociación del identificador de cámara `camera_num`` en el registro de la alarma), que permite al frontend Angular `web-core-view`` solicitar posteriormente la reproducción del segmento de vídeo grabado correspondiente al intervalo `[start_date, end_date]` de la alarma consultando directamente a `la-video-engine``.

---

## ## 7. Integración con el Frontend (web-core-view)

La interacción entre la aplicación Angular 19 (`web-core-view`) y el servicio de alarmas se basa en la persistencia del canal WebSocket. La aplicación web típicamente implementa un servicio Angular (`AlarmWebSocketService`) encargado de la reconexión y mapeo de eventos RxJS.

### ### Diagrama de Secuencia: Ciclo de Vida y Gestión de Eventos

El siguiente diagrama detalla cómo interactúa el frontend para visualizar alarmas en tiempo real y cómo realiza la gestión manual de los registros:

```
``mermaid
sequenceDiagram
 autonumber
 actor Operador
 participant Frontend as web-core-view (Angular 19)
 participant WS as la-core-alarm (WebSocket Server)
 participant ThreadCmd as la-core-alarm (Command Thread)
 participant ThreadDB as la-core-alarm (DB Client Thread)
 participant DBEngine as la-db-engine (gRPC Daemon)
 participant DBus as Bus D-Bus (Sistema)

 %% Establecimiento de Conexión
 Note over Frontend, WS: Inicialización de la Interfaz del Operador
 Frontend->>WS: Handshake WebSocket (Header Protocol: la-core-notifier)
 WS-->>Frontend: Conexión establecida (WebSocket Abierto)

 %% Flujo en Tiempo Real (Alarma del Sistema)
 Note over DBus, Frontend: Flujo 1: Notificación de Alarma del Sistema en Tiempo Real
 DBus->>WS: Señal ALARM_NOTIFY ("ssssssss")
 Note over WS: Callback de D-Bus captura señal
 WS->>ThreadDB: Pushes AlarmData a global_alarm_queue
 ThreadDB->>DBEngine: gRPC Execute (INSERT INTO ALARMS...)
 DBEngine-->>ThreadDB: Responde con last_insert_id (db_id)
 ThreadDB->>WS: Encola EmissionData a global_emission_queue
 Note over WS: lws_cancel_service() despierta bucle de envío
 WS->>Frontend: WebSocket Broadcast: DB_ENGINE_ALARM_NOTIFY
 Note over Frontend: Frontend actualiza el panel de alarmas en tiempo real

 %% Flujo de Consulta Histórica
 Note over Operador, Frontend: Flujo 2: Operador consulta el histórico de alarmas
 Operador->>Frontend: Solicita ver historial (scroll / paginación)
 Frontend->>WS: Envía mensaje WS: ALARMS_GET {select_date, quant}
 WS->>ThreadCmd: Encola CommandData a global_command_queue
 ThreadCmd->>DBEngine: gRPC Query (SELECT * FROM ALARMS WHERE start_date <
select_date...)
```



DBEngine-->>ThreadCmd: Retorna filas de la consulta (stream)  
ThreadCmd->>WS: Encola respuesta en cola del cliente  
WS->>Frontend: Envía mensaje WS: ALARMS\_GET\_SUCCESS [Array de alarmas]  
Note over Frontend: Renderiza rejilla histórica

%% Flujo de Operaciones (Marcar Leída)

Note over Operador, Frontend: Flujo 3: Operador marca alarma como leída  
Operador->>Frontend: Clic en "Marcar como Leída"  
Frontend->>WS: Envía mensaje WS: ALARM\_READ [{db\_id: 125, read: true}]  
WS->>ThreadCmd: Encola CommandData a global\_command\_queue  
ThreadCmd->>DBEngine: gRPC Execute (UPDATE ALARMS SET is\_read = 1 WHERE id = 125)  
DBEngine-->>ThreadCmd: Éxito (rows\_affected = 1)  
ThreadCmd->>WS: Encola respuesta en cola de salida del cliente  
WS->>Frontend: Envía mensaje WS: ALARM\_READ\_SUCCESS {db\_id: 125, read: true}  
Note over Frontend: Actualiza el estado visual de la alarma a leída  
...

### ### Protocolo de Integración en web-core-view

Cuando la aplicación Angular recibe una notificación de alarma ('DB\_ENGINE\_ALARM\_NOTIFY') o una respuesta de histórico ('ALARMS\_GET\_SUCCESS'), procesa los objetos JSON realizando la siguiente correspondencia y lógica interna:

1. **Identificador único:** Utiliza el campo `db\_id` del JSON para las operaciones de actualización local ('ALARM\_READ\_SUCCESS', 'ALARM\_DELETE\_SUCCESS').
2. **Reproducción del Fragmento de Vídeo Asociado:** Si `camera\_num` tiene un valor diferente a vacío, la interfaz Angular dibuja un botón de "Ver Grabación". Al pulsarlo, utiliza el intervalo `[start\_date, end\_date]` para construir la URL de reproducción en el reproductor de vídeo del frontend, enviando la petición de streaming al servicio `la-video-engine` (Mantos-Rec) por los protocolos de media oportunos (por ejemplo, WHEP/WebRTC o RTSP transpiled).
3. **Prioridad y Estilos:** El campo `priority` (típicamente `0`, `1`, `2`...) es utilizado por el sistema de hojas de estilo (CSS) del componente Angular para colorear y destacar las alarmas críticas (por ejemplo, parpadeo en color rojo corporativo de Lanaccess para prioridad alta, frente a tonalidades de amarillo o gris para eventos informativos).

la-core-notifier

## # Análisis Técnico del Servicio: `la-core-notifier`

Este documento detalla la arquitectura, el diseño de red, la gestión de memoria y el flujo de comunicación de la-core-notifier dentro del ecosistema del NVR (Network Video Recorder). Este servicio interactúa directamente con la aplicación frontend **web-core-view** (Angular 19) a través de canales WebSocket para proveer telemetría y eventos en tiempo real.

---

## ## 1. Resumen Ejecutivo y Propósito del Servicio

### ### Definición y Rol

El servicio `la-core-notifier` es un microservicio en red desarrollado en C++ cuyo rol principal es actuar como un **gateway de eventos en tiempo real** (Event Gateway). Sus principales responsabilidades son:

1. **Consumir eventos del sistema** emitidos por otros servicios del NVR (como `la-core-legacy`) a través del bus del sistema D-Bus.
2. **Consultar el estado actual global** de las entradas y salidas (E/S) del hardware a través de un segmento de memoria compartida (POSIX Shared Memory).
3. **Mantener conexiones WebSocket persistentes** con los clientes frontend (`web-core-view`) para enviar notificaciones selectivas del estado de sensores, relés, alarmas y eventos de cámaras (pérdida de señal, movimiento, analíticas, manipulación).

### ### Pila Tecnológica

- \* **Lenguaje:** C++20 (utilizando características como `std::jthread` para manejo automático de hilos, `std::stop_token` para detención segura y `std::format` para composición de cadenas).
- \* **Servidor WebSocket:** `libwebsockets` (LWS), una biblioteca en C ligera de alto rendimiento y bajo consumo para WebSocket e HTTP.
- \* **Manejo de JSON:** `jsoncpp` (`Json::Value`, `Json::Reader`, `Json::FastWriter`) para parseo y serialización de mensajes de control y eventos.
- \* **Integración del Sistema (IPC):**
  - \* `sd-bus` (parte de `systemd` / `libsystemd`) para la escucha reactiva de señales de bus.
  - \* Memoria Compartida POSIX (`shm_open`, `mmap`) para lecturas de estado rápidas con latencia ultra baja.

### ### Interacción con el Hardware/SO

El servicio no interactúa directamente con drivers físicos. Su relación con el sistema operativo y el hardware se delega en:

- \* **POSIX Shared Memory** (`/la_core_io_shm`): Lee una estructura mapeada en memoria física administrada por el NVR Core. Esto evita costosas consultas de E/S de disco o llamadas al sistema de red para conocer el estado físico de los periféricos.
- \* **Mecanismos de Bloqueo robustos en SHM:** Se utiliza un mutex robusto de proceso compartido (`pthread_mutex_t` con atributos `PTHREAD_PROCESS_SHARED` y `PTHREAD_MUTEX_ROBUST`) ubicado dentro de la estructura compartida para sincronizar las lecturas sin bloquear indefinidamente en caso de caída del proceso productor.

**\*\*D-Bus:\*\*** Suscribe filtros de señales D-Bus en ``/com/lanaccess/sysevents`` para recibir notificaciones instantáneas asíncronas sobre transiciones de E/S.

---

## ## 2. Arquitectura Interna y Patrones de Diseño

### ### Estructura de Directorios y Módulos

La organización del código del repositorio sigue un enfoque modular orientado a tareas:

...

la-core-notifier/

```
|— main.cpp # Punto de entrada, señalización y arranque de hilos principales.
|— CMakeLists.txt # Configuración de compilación CMake.
|— asyncapi.yaml # Especificación técnica del contrato WebSocket.
|— root/
| |— cfg/
| | |— include/cfgdim.h # Dimensiones límites del NVR (límites de cámaras según
build).
| |— core/
| | |— dbus/ # Hilo de escucha D-Bus.
| | |— tools/ # Convertidores de estado a texto humanizado.
| |— appnet/
| | |— websocket/ # Servidor WebSocket y procesadores de paquetes.
```

...

### ### Patrones de Diseño Aplicados

- \*\*Observer / Event-Driven Architecture:\*\*** Traduce el flujo interno de señales D-Bus a eventos en formato JSON propagados de forma masiva sobre conexiones TCP (WebSocket).
- \*\*Producer-Consumer (Thread-Safe Queue):\*\*** Cada conexión WebSocket activa almacena un objeto ``persistent_session_data`` que contiene un ``tools_queue<std::string>`` (una cola asíncrona basada en mutex y variables de condición). Los hilos externos (como el de D-Bus) producen mensajes que se inyectan en esta cola, mientras que el hilo del WebSocket Server los consume de manera no bloqueante.
- \*\*Resource Acquisition Is Initialization (RAII):\*\*** Se encapsula el ciclo de vida del contexto del servidor de libwebsockets en la clase ``lws_ctx_raii`` para garantizar la liberación de sockets y descriptores de archivos incluso ante excepciones.

### ### Gestión de Concurrencia y Estado

El programa inicializa varios hilos de ejecución mediante ``std::jthread``:

**\*\*Main Thread:\*\*** Configura los manejadores de señales (``SIGINT``, ``SIGTERM``) e inicializa el hilo de la lógica de aplicación (``Launcher``), durmiendo de manera segura en un ``std::condition_variable`` hasta la señal de apagado.

**\*\*WebSocket Thread (``web_socket_server_thread``):\*\*** Ejecuta de forma continua el bucle de eventos asíncrono de libwebsockets (``lws_service``).

**\*\*D-Bus Thread (``dbus_listen_la_core_legacy_events_thread``):\*\*** Utiliza la llamada de Linux ``poll()`` para monitorizar simultáneamente el descriptor de archivo de D-Bus y una

tubería (pipe) interna de cancelación (`global\_dbus\_cancel\_pipe\_fds`). Esto le permite reaccionar instantáneamente a señales de interrupción de D-Bus o notificaciones de apagado.

#### #### Memoria Compartida e Inter-Process Mutex

La estructura interna de datos en la memoria compartida está representada por `Entity\_SharedIOState`:

```
```cpp
struct Entity_IOEntry
{
    unsigned int idx;
    char        strTopic[80];
    unsigned int id;
    unsigned int idr;
    unsigned int data;
};

struct Entity_SharedIOState
{
    pthread_mutex_t    mutex;
    unsigned int        inputs_size;
    Entity_IOEntry      inputs[MAX_SHARED_INPUTS];
    unsigned int        virtual_inputs_size;
    Entity_IOEntry      virtual_inputs[MAX_SHARED_VIRTUAL_INPUTS];
    unsigned int        iv_inputs_size;
    Entity_IOEntry      iv_inputs[MAX_SHARED_IV_INPUTS];
    unsigned int        lava_inputs_size;
    Entity_IOEntry      lava_inputs[MAX_SHARED_LAVA_INPUTS];
    unsigned int        outputs_size;
    Entity_IOEntry      outputs[MAX_SHARED_OUTPUTS];
    unsigned int        remote_outputs_size;
    Entity_IOEntry      remote_outputs[MAX_SHARED_REMOTE_OUTPUTS];
#ifdef THERE_IS_VIDEO_PROCESSOR
    Entity_IOCamerEntry  camera[MAX_SHARED_CAMERAS];
#endif
    unsigned int        update_sequence_number;
};
```
```

Cuando un proceso se conecta, lee esta estructura y adquiere el mutex de pthread. Si un proceso muere mientras posee el lock, el siguiente proceso al adquirirlo recibe el código de error `EOWNERDEAD`. El código de `la-core-notifier` maneja esta contingencia ejecutando `pthread\_mutex\_consistent(&mapped\_shm\_state->mutex)` para reparar el mutex, evitando bloqueos permanentes.

---

### ## 3. Catálogo de Endpoints y APIs (REST / HTTP)

> [!NOTE]

> **Aclaración de Arquitectura:** El servicio `la-core-notifier` es un broker puramente de eventos en tiempo real y **no expone endpoints REST/HTTP**. No realiza operaciones de manipulación de ficheros, gestión de configuraciones, ni base de datos.

>

> Las peticiones HTTP tradicionales (GET/POST/PATCH para configuraciones de usuarios, I/O, etc.), el sistema de bloqueo pesimista (`acquireLock`), y la generación de esquemas son gestionados por el servicio principal `la-core-legacy` (en el puerto HTTP estándar). `la-core-notifier` solo opera en el puerto TCP `60070` usando la conexión de protocolo WebSocket `"la-core-notifier"`.

---

### ## 4. Protocolos en Tiempo Real (WebSockets)

#### ### Endpoints de Conexión

**Puerto de Escucha Interno:** `60070`

**Protocolo WebSocket (Subprotocolo LWS):** `la-core-notifier`

**Path de Red (Expuesto al Frontend a través de Nginx):**

`ws://<nvr_ip>/ws/la-core-notifier`

---

#### ### Estructura de Mensajes (JSON)

##### #### 1. Cliente al Servidor: Actualización de Filtros (`filter_update`)

El frontend Angular envía este mensaje para suscribirse selectivamente a prefijos de temas de eventos (tópicos).

**Payload de Entrada (JSON):**

```
``json
{
 "type": "filter_update",
 "transactionId": "tx_req_1001",
 "payload": {
 "include_topics": ["io/input", "io/camera/motion"],
 "exclude_topics": ["io/camera/motion/2"]
 }
}
```

...

**Campos:**

\* `type` (Obligatorio, string): Debe ser `"filter_update"`.

\* `transactionId` (Opcional, string): ID proporcionado por el cliente para trazar la respuesta asíncrona.

\* `payload` (Obligatorio, objeto):

\* `include_topics` (Array de strings): Prefijos de tópicos a los que suscribirse.

\* `exclude\_topics` (Array de strings): Prefijos de tópicos específicos a excluir de la suscripción anterior.

#### #### 2. Servidor al Cliente: Confirmación del Filtro (`ack`)

\* \*\*Payload de Salida (JSON):\*\*

```
```json
{
  "event": "ack",
  "transactionId": "tx_req_1001",
  "message": "Filtros actualizados correctamente."
}
```
```

#### #### 3. Servidor al Cliente: Error en Petición (`error`)

\* \*\*Payload de Salida (JSON):\*\*

```
```json
{
  "event": "error",
  "transactionId": "tx_req_1001",
  "message": "Campo 'payload' faltante o inválido en la solicitud filter_update."
}
```
```

#### #### 4. Servidor al Cliente: Notificación de Estado Inicial (`IO\_NOTIFY\_INIT`)

Enviado automáticamente de forma asíncrona al cliente inmediatamente después de un `filter\_update` exitoso. Contiene el estado de todos los recursos del NVR que cumplen con los filtros de suscripción activos.

\* \*\*Payload de Salida (JSON):\*\*

```
```json
{
  "event": "IO_NOTIFY_INIT",
  "payload": {
    "io/input": [
      { "id": "1", "idr": "101", "data": "idle" },
      { "id": "2", "idr": "102", "data": "on" }
    ],
    "io/output": [
      { "id": "1", "idr": "201", "data": "deactivated" }
    ],
    "io/camera/motion": [
      { "id": "3", "idr": "3", "data": "idle" },
      { "id": "4", "idr": "4", "data": "recording-alarm+watching" }
    ]
  }
}
```
```

\* \*\*Valores de Estado Dinámicos (`data`):\*\*

```

* **Digital Inputs (`io/input`, `io/virtual_input`, etc.):**
* `idle` (Reposo)
* `on` (Activa)
* `open-circuit` (Sabotaje / circuito abierto)
* `short-circuit` (Cortocircuito)
* `disabled` (Deshabilitado)
* `disarmed` (Desarmado)
* `not-configured` (Sin configurar)
* `unknown` (Desconocido)
* **Digital Outputs (`io/output`, `io/remote_output`):**
* `activated` (Relé cerrado)
* `deactivated` (Relé abierto)
* `not-configured` (Sin configurar)
* **Camera States (`io/camera/no-signal`, `io/camera/motion`, `io/camera/tamper`,
`io/camera/analytics`):**
* `no-signal` (Pérdida de señal de vídeo)
* `pre-recording` (Pre-grabación)
* `recording-alarm` (Grabación por alarma activa)
* `post-recording` (Post-grabación activa)
* `recording-continous` (Grabación continua programada)
* `recording` (Grabando genérico)
* `watching` (Visualización activa)
* `idle` (Cámara en reposo / sin eventos)

```

##### 5. Servidor al Cliente: Notificaciones de Eventos en Tiempo Real (`IO\_NOTIFY`)  
Generadas dinámicamente cuando el backend recibe una señal de D-Bus de cambio de estado de E/S.

```

* **Payload de Salida (JSON):**
```json
{
  "event": "IO_NOTIFY",
  "payload": [
    {
      "topic": "io/input",
      "id": "2",
      "idr": "102",
      "time": "2026-06-10T12:00:00Z",
      "data": "idle"
    }
  ]
}
```

```

##### 6. Servidor al Cliente: Errores de Motor / Hardware (`ERRORS\_NOTIFY`)  
Generado cuando ocurre un problema crítico a nivel de NVR core.

```

* **Payload de Salida (JSON):**

```



```

```json
{
  "event": "ERRORS_NOTIFY",
  "payload": [
    {
      "errors": "5",
      "pending-errors": "2",
      "topic": "device/notification"
    }
  ]
}
```

```

#### #### 7. Servidor al Cliente: Actualización de Software (`SOFTWARE\_UPDATE\_NOTIFY`)

\* \*\*Payload de Salida (JSON):\*\*

```

```json
{
  "event": "SOFTWARE_UPDATE_NOTIFY",
  "payload": [
    {
      "topic": "device/notification"
    }
  ]
}
```

```

---

#### ### Algoritmo de Filtrado Dinámico de Temas (Tópicos)

El servidor valida y filtra dinámicamente qué mensajes de red se envían a cada socket utilizando los arrays `included\_topic\_prefixes` y `excluded\_topic\_prefixes`. El algoritmo implementa las siguientes reglas:

```

```mermaid

```

flowchart TD

```

    Start([Recibir evento del sistema]) --> GetClientFilters[Obtener filtros de sesión del Cliente]

```

```

    GetClientFilters --> CheckIncludesEmpty{¿include_topics está vacío?}

```

```

    CheckIncludesEmpty -- Sí --o CheckExcludesEmpty{¿exclude_topics está vacío?}

```

```

    CheckExcludesEmpty -- Sí --o DropItem[Filtrar: No enviar evento]

```

```

    CheckExcludesEmpty -- No --o MatchExcludesPrefixes{¿Coincide con algún prefijo en exclude_topics?}

```

```

    CheckIncludesEmpty -- No --o MatchIncludesPrefixes{¿Coincide con algún prefijo en include_topics?}

```

```

    MatchIncludesPrefixes -- No --o DropItem

```

```

    MatchIncludesPrefixes -- Sí --o MatchExcludesPrefixes

```

```

    MatchExcludesPrefixes -- Sí --o DropItem
    MatchExcludesPrefixes -- No --o AcceptItem[Aceptar: Añadir al JSON de salida]
    ...

* **Código de validación en C++:**
    El método `check_topic_against_filters` comprueba si el tema del evento comienza con
    alguno de los prefijos de la lista de inclusión y descarta explícitamente aquellos que
    comiencen con elementos de la lista de exclusión.

    ---

### Máquina de Estados de Conexión

``mermaid
stateDiagram-v2
    [*] --> Listening : Servidor arranca en puerto 60070
    Listening --> ConnectionRequested : Cliente inicia handshake WebSocket

    ConnectionRequested --> ConnectionRejected : Conexiones activas >= 10
    ConnectionRejected --> [*]

    ConnectionRequested --> ConnectionEstablished : Conexiones activas < 10
    state ConnectionEstablished {
        [*] --> AwaitingFilterUpdate : Conexión abierta (WSI registrado)
        AwaitingFilterUpdate --> ActiveSubscription : Recibe 'filter_update' válido \n(envía ACK
+ IO_NOTIFY_INIT)
        ActiveSubscription --> ActiveSubscription : Recibe D-Bus Signal (evalúa filtros e
interactúa)
        ActiveSubscription --> AwaitingFilterUpdate : Recibe nuevo 'filter_update' (limpia y
actualiza)
    }

    ConnectionEstablished --> ConnectionClosed : Cliente desconectado / Network Loss
\n(LWS_CALLBACK_CLOSED)
    ConnectionClosed --> [*] : Libera datos persistentes en el mapa
    ...

* **Límite de Conexiones:** Hardcodeado a `MAX_CONCURRENT_CONNECTIONS = 10`.
    Esto protege al NVR de ataques de denegación de servicio (DoS) que agoten los
    descriptores de archivos del kernel.
* **Keep-Alive:** No hay lógica Ping/Pong a nivel de aplicación JSON. Se aprovecha el
    control nativo PING/PONG a nivel de protocolo WebSocket provisto por `libwebsockets` de
    manera transparente para el sistema operativo.

    ---

## 5. Mecanismo de Configuración, Bloqueo y Esquemas (Delegado)

```

Como se ha especificado, la gestión activa de configuraciones y bloqueos no pertenece a `la-core-notifier`. Sin embargo, para contextualizar tu TFG:

1. **Pessimistic Locking** (`acquireLock` / `releaseLock`): Es ejecutado por **la-core-legacy**. El frontend Angular realiza un GET solicitando un `token` y un `seed`. Las siguientes peticiones REST que mutan el sistema deben adjuntar este token como query parameter. Si otra sesión de usuario intenta editar, el backend responde con un código de error `409 Conflict`.
2. **Patch Merging y Reinicio de Hardware**: Cuando `la-core-legacy` recibe un PATCH de configuración de cámara o red, realiza un merge selectivo. Si el backend detecta cambios en propiedades estáticas (como IPs de red o contraseñas físicas de streams RTSP), marca la bandera `\_rebootRequired` internamente en su objeto de configuración y añade la cabecera HTTP `X-Reboot-Required` en su respuesta para alertar al frontend Angular.

6. Pipeline de Vídeo, Almacenamiento o Telemetría (Delegado)

El pipeline de ingesta es administrado por **la-video-engine**:

Gestión de Streams: Procesa códecs H.264/H.265 para su empaquetado en WebRTC (WHEP) y HLS.

Flujo de Eventos: Cuando `la-video-engine` detecta un corte de señal o un evento de analítica de vídeo, emite un mensaje formateado a través del bus del sistema D-Bus (`com.lanaccess.sysevents`).

Rol de Notifier en el Pipeline: `la-core-notifier` captura dicho evento D-Bus asíncronamente y lo transmite a `web-core-view` de inmediato en formato JSON (`IO\_NOTIFY` con tópicos `io/camera/motion` o `io/camera/no-signal`), permitiendo que el visor del operador dibuje la alerta visual en milisegundos.

7. Integración con el Frontend (web-core-view)

A continuación, se detalla el flujo de interacción completo entre la aplicación frontend en Angular 19, el servidor de eventos `la-core-notifier` y el Core Legacy del NVR:

```
``mermaid
sequenceDiagram
    autonumber
    actor Operador
    participant Angular as web-core-view (Angular 19)
    participant Notifier as la-core-notifier (C++ / 60070)
    participant Core as la-core-legacy (D-Bus Publisher)
    participant SHM as POSIX Shared Memory (/la_core_io_shm)

    %% Establecimiento de la sesión en tiempo real
    Operador->>Angular: Carga panel de cámaras/sensores
    Angular->>Notifier: ws_connect() a ws://NVR_IP/ws/la-core-notifier
```

Notifier->>Notifier: Evalúa conexiones activas (limite 10)
Notifier-->>Angular: LWS_CALLBACK_ESTABLISHED (Conexión abierta)

%% Configuración de filtros

Angular->>Notifier: filter_update (include: ["io/input", "io/camera/motion"])

Notifier->>Notifier: Actualiza listas de prefijos de sesión

Notifier-->>Angular: ack (Filtros configurados con éxito)

%% Estado inicial

Notifier->>SHM: pthread_mutex_lock()

SHM-->>Notifier: Retorna puntero mapped struct

Notifier->>Notifier: Filtra tópicos y extrae estados actuales de E/S

Notifier->>SHM: pthread_mutex_unlock()

Notifier-->>Angular: IO_NOTIFY_INIT (JSON con todos los estados del filtro)

Angular->>Operador: Renderiza interfaz con estados actualizados (Sensores en Verde/Rojo)

%% Recepción de eventos en tiempo real

Note over Core, SHM: Sensor físico de puerta cambia a ON

Core->>SHM: pthread_mutex_lock() & Actualiza entrada 'data = 1'

Core->>SHM: pthread_mutex_unlock()

Core->>Notifier: D-Bus Signal: IO_NOTIFY (Array con id="2", topic="io/input", data="on")

Notifier->>Notifier: Analiza evento recibido en sd-bus loop

Notifier->>Notifier: Compara "io/input" contra filtros de clientes activos

Notifier-->>Angular: IO_NOTIFY (JSON con nuevo estado "on" para id="2")

Angular->>Operador: Cambia dinámicamente icono de puerta a color rojo en la interfaz reactiva
...

Este modelo desacoplado permite que el NVR atienda miles de transiciones internas por segundo con latencia mínima, garantizando al mismo tiempo que el navegador web de Angular reciba únicamente la información que le interesa visualizar de forma reactiva mediante WebSockets.

mantos-rec

Análisis de Arquitectura y APIs: NVR Core Services

Este documento contiene un análisis técnico exhaustivo del backend de nuestro Network Video Recorder (NVR) y su interacción con el frontend desarrollado en Angular (`web-core-view`). Este informe está estructurado específicamente para servir como base documental en el Trabajo de Fin de Grado (TFG).

1. Resumen Ejecutivo y Propósito de los Servicios

El backend del NVR en este repositorio (gestionado por la receta de Yocto `mantos-rec`) no es un único servicio monolítico, sino una arquitectura distribuida de microservicios e IPCs especializados que corren sobre una imagen Linux empotrada a medida (basada en el BSP de NXP para i.MX8). Sus principales componentes son:

- * **`la-core-legacy` (Servicio Central):** Orquestador central del sistema. Implementa el servidor HTTP/REST principal, maneja la autenticación y sesiones de usuario, gestiona el reloj del sistema, las licencias y actúa como pasarela D-Bus para los cambios de configuración.
- * **`la-core-notifier` (Gateway de Eventos E/S):** Servidor de WebSockets enfocado exclusivamente a notificar cambios de estado en tiempo real de entradas/salidas físicas y virtuales, así como telemetría de cámaras (pérdida de señal, movimiento, manipulación y analíticas).
- * **`la-core-alarm` (Gestión de Alarmas):** Servidor de WebSockets especializado en el ciclo de vida de las alarmas registradas en la base de datos (lectura, archivado, borrado y filtrado temporal).
- * **`la-video-engine` (Motor de Medios):** Núcleo C++ que interactúa con GStreamer y encapsula el pipeline de ingesta de vídeo de cámaras IP, grabación continua/eventos en disco y streaming de reproducción. Integra **MediaMTX** como servidor de streaming final.
- * **`la-db-engine` (Motor de Base de Datos):** Gestiona las bases de datos locales SQLite/propietarias donde se almacena el histórico de alarmas, logs y el estado persistente.

Pila Tecnológica

- * **Lenguajes:** C++17/C++20 para todos los servicios de bajo nivel y lógica de negocio core.
- * **Servidor REST/HTTP:** Basado en `libmicrohttpd` y wrappers CGI en C++ (`cgi\_get\_strpar`, `cgi\_get\_grant`).
- * **WebSockets:** Desarrollados sobre `libwebsockets` y serialización JSON mediante `jsoncpp`.
- * **Base de datos:** Motor propietario de almacenamiento de metadatos de vídeo combinada con SQLite para índices relacionales rápidos de alarmas.
- * **IPC:** D-Bus del sistema (`dbus-1`) como bus principal para la comunicación inter-proceso (ej. `la-core-legacy` escribe a través de D-Bus y `la-video-engine` o `la-system-properties-validator` reaccionan).

Interacción con el Hardware / SO

- * ****Módulos de E/S Física:**** El servicio interactúa con el driver GPIO del kernel i.MX8 mediante ``libgpiod`` para leer entradas digitales físicas y conmutar relés de salida.
- * ****Discos Físicos (SATA/NVMe):**** Interacción directa a nivel de bloque y sistema de archivos POSIX. Para la grabación, se preasignan archivos circulares grandes para evitar fragmentación. El motor monitoriza las temperaturas e información S.M.A.R.T. de los discos.
- * ****Configuración de Red:**** Modifica ficheros ``config_network_if_eth0.yml`` y aplica configuraciones dinámicas de interfaces (ej. ``end0``, ``end1`` en i.MX8) a través de ``systemd-networkd``.

2. Arquitectura Interna y Patrones de Diseño

El sistema utiliza varios patrones de diseño orientados a sistemas empujados de alto rendimiento y robustez:

Estructura de Módulos (la-core-legacy)

...

la-core-legacy/root/

```

├── appnet/      # Capa de red y protocolos
├──   └── api-rest/ # Definición de rutas, controladores de endpoints REST
├── cfg/        # Lógica y buffers de configuración en disco (.crypt, .yaml)
├── core/       # Clases core del sistema, sesión y permisos
├── video/      # Clientes IP (ONVIF, Hikvision, etc.) para control secundario
└── webinteface/ # Lógica de wizards, reports y configuraciones heredadas (CGI)

```

...

Patrones de Diseño Clave

1. ****REST Tree-Router (Enrutador Jerárquico estático):****

Los endpoints REST se definen estáticamente como nodos en un árbol (``Entity_ApiRestElementNodeTree`` / ``Entity_ApiRestTokenNodeTree``). El router recorre este árbol comparando segmentos del URI para delegar en el callback ``DECLARE_API_REST_FUNC`` adecuado, validando permisos (grants) antes de ejecutar el callback.

2. ****Pipeline Manager (la-video-engine):****

Usa el patrón Pipeline de GStreamer. Encapsula la gestión de recursos de vídeo mediante contextos (``receiver_pipeline_contexts``) y factorías de reproducción (``rtsp_server_pipeline_contexts``).

3. ****Active Lock Registry (g_active_locks):****

Patrón Singleton de concurrencia que mantiene un mapa thread-safe de bloqueos sobre nodos de configuración para evitar colisiones de edición simultánea desde múltiples sesiones del Angular.

Gestión de Concurrencia y Estado

El backend es altamente concurrente, implementando un pool de hilos para conexiones HTTP, un hilo dedicado para el event-loop de ``libwebsockets`` y múltiples hilos en ``la-video-engine`` para procesar flujos multimedia.

- * ****Estado en Memoria:**** El estado de configuración del NVR se mantiene en un árbol plano clave-valor (``current_properties``).
- * ****Mutexes:**** ``g_locks_mutex`` sincroniza la adquisición y expiración de locks en ``la-core-legacy``. ``rtspServerPipelineContextsMutex`` gestiona los accesos concurrentes a las conexiones RTSP.

3. Catálogo de Endpoints y APIs (REST / HTTP)

La API REST corre por defecto bajo el servicio central ``la-core-legacy`` (puerto 80/443).

GET ``/api/v1/device/properties/schema?node=<path>``

- * ****Propósito:**** Obtiene el esquema de validación JSON Schema para un nodo concreto de configuración (ej. ``config.video.cam.000``).
- * ****Query Params:**** ``node`` (Obligatorio - String).
- * ****Respuesta (200 OK):****

```
```json
{
 "config.video.cam.000": {
 "type": "object",
 "properties": {
 "enabled": { "type": "boolean", "default": true },
 "name": { "type": "string", "maxLength": 32 },
 "ip": { "type": "string", "format": "ipv4" },
 "port": { "type": "integer", "_default": 80, "_rebootRequired": true }
 }
 }
}
```
```

GET ``/api/v1/device/properties/template?node=<path>``

- * ****Propósito:**** Devuelve una plantilla de inicialización del nodo con los valores por defecto definidos en su esquema.
- * ****Query Params:**** ``node`` (Obligatorio).
- * ****Respuesta (200 OK):****

```
```json
{
 "enabled": true,
 "name": "",
 "ip": "0.0.0.0",
 "port": 80
}
```
```

GET ``/api/v1/device/properties/data?node=<path>``

- * ****Propósito:**** Devuelve los valores de configuración actuales del nodo y sus descendientes.

* **Query Params:** ``node`` (Obligatorio).

* **Respuesta (200 OK):**

```
```json
{
 "enabled": true,
 "name": "Camara Pasillo",
 "ip": "192.168.1.50",
 "port": 8080
}
```

#### ##### PATCH

``/api/v1/device/properties/data?node=<path>&token=<lockToken>&seed=<clientSeed>``

\* **Propósito:** Actualiza de forma parcial o masiva un nodo. Soporta dos modos:

1. **Parche Estándar:** Modificaciones directas en formato JSON clave-valor.
2. **RFC 6902 JSON Patch:** Enviando cabecera ``Content-Type``:

`application/json-patch+json``.

\* **Query Params:** ``node``, ``token``, ``seed``.

\* **Payload (JSON Patch con Wildcards y Filtro Posicional):**

```
```json
[
  {
    "op": "replace",
    "path": "/config/video/cam/{}/port",
    "where": "enabled==true",
    "value": 80
  }
]
```

*Nota: El motor expande ``{}`` buscando todos los IDs de cámara en el árbol de datos y evalúa la expresión de filtrado del atributo ``where`` (``enabled==true``).

* **Respuesta (200 OK):** Retorna el estado consolidado del nodo tras aplicar el parche (equivalente a un GET).

Manejo de Errores y Códigos de Estado

El backend devuelve respuestas estructuradas en caso de fallo:

* **400 Bad Request (Error de Validación):** Cuando los datos no cumplen con las restricciones del JSON Schema (ej. IP mal formateada).

```
```json
{
 "action": "Sender",
 "cause": "ValidationError",
 "validationErrors": [
 {
 "property": "config.video.cam.000.ip",
 "reason": "Does not match IPv4 format pattern"
 }
]
}
```

```
]
}
```

\* **403 Forbidden (Fallo de Lock o Privilegios):** Cuando se intenta escribir sin token, con un token incorrecto, o el usuario no tiene la máscara de permisos necesaria (``grants``).

```
```json
{
  "action": "Sender",
  "cause": "PrivilegeError"
}
```

* O si es por desajuste de bloqueo: *

```
```json
{
 "action": "Sender",
 "cause": "UnlockFailed",
 "detail": "Invalid token for the specified lock."
}
```

\* **409 Conflict (Recurso Bloqueado):** Cuando se intenta adquirir un bloqueo en un path ya bloqueado por otra IP o usuario.

```
```json
{
  "action": "Sender",
  "cause": "ResourceLocked",
  "detail": "Resource is already locked by user 'admin' from IP 192.168.1.100",
  "lockedPath": "config.video"
}
```

4. Protocolos en Tiempo Real (WebSockets)

El sistema expone dos pasarelas WebSocket críticas a través del proxy inverso Nginx (`/ws/...``):

Gateway 1: ``la-core-notifier``

* **Ruta de conexión:** ``/ws/la-core-notifier``

* **Mensaje del Cliente al Servidor (``filter_update``):**

Permite al frontend suscribirse solo a tópicos específicos para ahorrar ancho de banda:

```
```json
{
 "type": "filter_update",
 "transactionId": "tx-10294",
 "payload": {
 "include_topics": ["io/input", "io/camera/motion"],
 "exclude_topics": []
 }
}
```

```
}
}
...
```

\* \*\*Mensajes del Servidor al Cliente:\*\*

1. `IO\_NOTIFY\_INIT` (Estado inicial al conectar):

```
```json  
{  
  "event": "IO_NOTIFY_INIT",  
  "payload": {  
    "io/input": [  
      { "id": "input.00", "idr": "in0", "data": "0", "topic": "io/input" }  
    ],  
    "io/camera/motion": [  
      { "id": "camera.00.motion", "idr": "mot0", "data": "1", "topic": "io/camera/motion" }  
    ]  
  }  
}  
```
```

2. `IO\_NOTIFY` (Cambio en caliente de E/S):

```
```json  
{  
  "event": "IO_NOTIFY",  
  "payload": [  
    {  
      "topic": "io/input",  
      "id": "input.00",  
      "idr": "in0",  
      "time": "2026-06-10T12:40:00Z",  
      "data": "1"  
    }  
  ]  
}  
```
```

3. `ERRORS\_NOTIFY` (Notificaciones de errores de hardware/software):

```
```json  
{  
  "event": "ERRORS_NOTIFY",  
  "payload": [  
    { "errors": "DiskTemperatureHigh", "pending-errors": "1", "topic": "system/errors" }  
  ]  
}  
```
```

##### Gateway 2: `la-core-alarm`

\* \*\*Ruta de conexión:\*\* `/ws/la-core-alarm`

\* \*\*Mensaje de suscripción/comandos del Cliente:\*\*

El cliente puede marcar como leídas, archivar o solicitar alarmas:

```
```json
```

```
{
  "event": "ALARM_READ",
  "payload": [
    { "db_id": "28349", "read": true }
  ]
}
...
```

* **Mensajes del Servidor al Cliente:**

1. `DB\_ENGINE\_ALARM\_NOTIFY` (Alarma en tiempo real disparada por analítica o E/S):

```
```json
{
 "event": "DB_ENGINE_ALARM_NOTIFY",
 "payload": [
 {
 "alarm_id": "99",
 "db_id": "28350",
 "description": "Cruze de línea - Entrada Principal",
 "priority": "alta",
 "source": "Camara 01",
 "start_date": "2026-06-10T12:41:00Z",
 "end_date": "",
 "status": "active",
 "timestamp": "1773229260",
 "topic": "alarm",
 "type": "analytics",
 "camera_num": "0",
 "is_read": false,
 "is_archived": false,
 "is_locked": false
 }
]
}
...
```

#### #### Gestión del Ciclo de Vida del WebSocket

\* \*\*Ping/Keep-alive:\*\* Se gestiona de manera transparente por `libwebsockets` mediante tramas PING/PONG a nivel RFC 6455.

\* \*\*Reconexión:\*\* Si el socket se cierra de forma inesperada, el frontend de Angular detecta la desconexión e inicia un reintento exponencial (`backoff`) cada \$2n\$ segundos para evitar saturar el puerto del servidor.

---

### ### 5. Mecanismo de Configuración, Bloqueo y Esquemas

La integridad de la configuración distribuida del NVR se garantiza a través de un mecanismo estricto de concurrencia y validación jerárquica.

#### #### Sistema de Bloqueo Pesimista (Pessimistic Locking)

Cuando un usuario del frontend intenta modificar cualquier nodo de configuración, se debe establecer un bloqueo sobre dicho nodo:

```
```mermaid
```

```
sequenceDiagram
```

```
    autonumber
```

```
    participant FE as Angular Frontend (web-core-view)
```

```
    participant BE as Core Legacy (REST API)
```

```
    FE->>BE: GET /properties/lock?node=config.video.cam&seed=abc1234
```

```
    Note over BE: Valida que ningún padre (ej. config)<br/>ni hijo (ej. config.video.cam.00) esté bloqueado.
```

```
    BE-->>FE: 200 OK {token: "32_char_alphanumeric", expiresIn: 120}
```

```
    Note over FE: El usuario realiza cambios locales
```

```
    FE->>BE: PATCH
```

```
    /properties/data?node=config.video.cam&token=32_char_...&seed=abc1234
```

```
    Note over BE: Valida token, seed, IP y usuario.<br/>Renueva automáticamente lock por otros 120s.
```

```
    BE-->>FE: 200 OK {consolidated_data}
```

```
    FE->>BE: GET
```

```
    /properties/unlock?node=config.video.cam&token=32_char_...&seed=abc1234
```

```
    Note over BE: Libera la clave del mapa g_active_locks
```

```
    BE-->>FE: 200 OK
```

```
    ...
```

* **Conflictos Jerárquicos:**

* **Parent-locked:** Si pido un lock sobre `config.video.cam.00` pero `config.video` está bloqueado por otro usuario, se deniega (409 Conflict).

* **Child-locked:** Si pido un lock sobre `config` pero un usuario está configurando `config.video.cam.00`, se deniega (409 Conflict).

* **Renovación automática:** Si el dueño del lock hace peticiones de lectura/escritura sobre el mismo nodo, el tiempo de expiración se extiende otros 120 segundos.

Motor de Esquemas y Plantillas

* **`/properties/schema`:** Obtiene el esquema directamente consultando por IPC local a `la-system-properties-validator` (el cual lee los ficheros YML compilados en el firmware).

* **`/properties/template`:** `la-core-legacy` genera esto dinámicamente:

1. Carga el JSON Schema del nodo deseado.
2. Instancia un árbol de esquema interno en C++ (`SchemaNode`).
3. Crea un árbol de datos vacío (`DataNode`).
4. Llama a `merge\_trees(data, schema, indices, true)` para poblar automáticamente los nodos hijos y los valores por defecto del esquema (`\_default`).

Fusión de Cambios (Patch Merging) e Inicio Inteligente

1. ****Parcheo:**** Al recibir un `PATCH` (o un array de operaciones RFC 6902), se valida el input sintácticamente contra el JSON Schema.
2. ****D-Bus Transaction:**** Si pasa la validación, la-core-legacy convierte el JSON plano a una lista de propiedades planas estructurada y la envía a través de D-Bus (`dbus_querier_call_set_list_of_properties``).
3. ****Detección Inteligente de Reinicios:****

El sistema compara el nuevo valor con el valor actual o el valor por defecto. Si el valor cambia y en su esquema la propiedad tiene la propiedad ``_rebootRequired: true``, el backend ejecuta ``system_mark_reboot_required()``. Esto provoca que las siguientes respuestas REST incluyan la cabecera HTTP ``X-Reboot-Required: true``, indicando al frontend de Angular que debe mostrar una alerta visual de reinicio pendiente.

6. Pipeline de Vídeo, Almacenamiento y Telemetría

Gestión de Flujos de Vídeo (la-video-engine)

- * ****Ingesta de Medios:**** El motor C++ crea pipelines de GStreamer con elementos ``rtspsrc`` para conectarse a las cámaras IP y extraer las pistas de audio y vídeo H.264/H.265.
- * ****Streaming de Salida:****
 - * ****RTSP:**** Expuesto a través de ``gst-rtsp-server`` para la integración clásica con otros VMS.
 - * ****HLS / WebRTC (WHEP):**** Mediante la integración estrecha de ****MediaMTX****. El motor genera streams RTSP locales que se publican en el MediaMTX interno. MediaMTX fragmenta en ``ts`` y genera los archivos index ``m3u8`` (HLS) o sirve sesiones WebRTC usando señalización WHEP para reproducción en el navegador sin latencia.

Estructura de Almacenamiento Físico

El almacenamiento de vídeo gestiona la grabación circular sobre múltiples discos duros según los modos RAID configurados en ``DEVICE_SERVICE_HD_RAID_MODE``:

- * ****`disabled`:**** Los discos duros se tratan de forma independiente. Si uno falla, solo se pierden las grabaciones de las cámaras asignadas a ese disco.
- * ****`mirror` (RAID 1):**** Espejo en caliente de 2 discos duros. El sistema replica todas las operaciones de escritura y soporta la reconstrucción progresiva tras sustituir un disco degradado.
- * ****`double-mirror`:**** Replicación en espejo sobre 3 o más discos para mayor tolerancia a fallos.
- * ****`big` (JBOD):**** Concatenación lógica de todos los discos para formar un único volumen gigante de almacenamiento circular.
- * ****`n+1`:**** Asignación dinámica de un disco de reserva caliente (``spare``). Si un volumen activo falla, el motor re-enruta el pipeline de grabación al disco spare de forma automática.

Métricas de Rendimiento (``/video-engine/stats.json``)

El backend expone métricas internas estructuradas en las siguientes áreas de ingeniería:

Grupo	Métricas Clave	Propósito en Ingeniería
:---	:---	:---

| **Message Queue** | `Depth`, `Batch Size`, `Total Pushes/Pops`, `Drops` | Evalúa si el hilo consumidor está procesando tramas de vídeo a tiempo o si la cola de GStreamer se está desbordando (Frame drops). |

| **Buffer Arena** | `2MB/4MB/8MB Pool Capacity`, `OS Allocations (mmap)` | Monitoriza la fragmentación y la asignación de memoria RAM física del SoC i.MX8 para evitar desbordamientos de OOM (Out Of Memory). |

| **Disks** | `Reads/Writes`, `unaligned\_reads/writes`, `open/close errors` | Analiza el rendimiento físico de escritura. Las escrituras desalineadas (`unaligned`) reducen drásticamente la vida útil de los discos duros mecánicos de videovigilancia. |

| **Recorder** | `State`, `GOP Size`, `Fill Time`, `Fill Percentage`, `Missed I-Frame` | Estado de la grabación. Un GOP irregular o la pérdida de frames tipo I (`Missed I-Frame`) impiden el indexado correcto y degradan la calidad del streaming. |

7. Flujo de Integración con el Frontend (web-core-view)

A continuación se muestra el diagrama de secuencia lógico que describe la comunicación completa en un ciclo de uso real de la aplicación Angular:

```

sequenceDiagram
    participant FE as Angular web-core-view
    participant BE_Core as la-core-legacy
    participant BE_WS as la-core-notifier / la-core-alarm
    participant BE_Video as la-video-engine

    Note over FE, BE_Core: 1. AUTENTICACIÓN
    FE->>BE_Core: POST /api/v1/login {User, Password}
    BE_Core-->>FE: 200 OK (Crea sesión en cookie/cabecera)
    FE->>BE_Core: GET /api/v1/whoami
    BE_Core-->>FE: 200 OK {user: "admin", profile: "configurador", grants: "0xFFFF"}

    Note over FE, BE_WS: 2. CONEXIÓN EN TIEMPO REAL
    FE->>BE_WS: Handshake WebSocket /ws/la-core-notifier
    BE_WS-->>FE: Conexión Establecida
    BE_WS-->>FE: Evento IO_NOTIFY_INIT (Estados iniciales de E/S y cámaras)
    FE->>BE_WS: filter_update {include_topics: ["io/input", "io/camera/motion"]}

    Note over FE, BE_Core: 3. MODIFICACIÓN DE CONFIGURACIÓN
    FE->>BE_Core: GET /properties/lock?node=config.video.cam.00&seed=xyz
    BE_Core-->>FE: 200 OK {token: "tok_abc123"}
    FE->>BE_Core: PATCH /properties/data?node=config.video.cam.00&token=tok_abc123
    {name: "Nueva Cámara"}

    Note over BE_Core: Lógica de validación de esquemas y escritura DBus
    BE_Core-->>FE: 200 OK (Datos actualizados) + Cabecera X-Reboot-Required (opcional)
    FE->>BE_Core: GET /properties/unlock?node=config.video.cam.00&token=tok_abc123
  
```

Note over BE_WS, FE: 4. EVENTO DE ALARMA Y STREAMING

BE_WS-->>FE: Evento DB_ENGINE_ALARM_NOTIFY (Nueva Alarma en cámara 00)

FE->>BE_Video: Request WHEP Stream /ws-webrtc/camera00

BE_Video-->>FE: SDP Exchange (WebRTC sub-second latency rendering)

...

prompt

Hola. Estoy documentando la arquitectura de nuestro sistema NVR (Network Video Recorder) para mi Trabajo de Fin de Grado (TFG). Necesito que realices un análisis técnico profundo de este repositorio correspondiente al servicio: [Insertar Nombre del Servicio].

Dado que este backend interactúa directamente con nuestra aplicación frontend de control ("web-core-view", desarrollada en Angular 19), necesito una descripción exhaustiva y estructurada de su diseño, API, eventos y lógica interna.

Por favor, genera un informe técnico estructurado en las siguientes secciones:

1. Resumen Ejecutivo y Propósito del Servicio

- **Definición y rol:** ¿Cuál es la responsabilidad principal de este servicio dentro del ecosistema del NVR?
- **Pila tecnológica:** Lenguaje de programación, frameworks, base de datos (si la hay) o archivos de configuración locales que maneja.
- **Interacción con el hardware/SO:** ¿Este servicio interactúa directamente con drivers del sistema, sistemas de archivos de disco, controladores de red o módulos físicos de Entrada/Salida (I/O)? Explica brevemente cómo.

2. Arquitectura Interna y Patrones de Diseño

- **Patrón arquitectónico:** Describe la estructura de directorios/módulos de este repositorio y los patrones aplicados (ej. MVC, microservicios, controladores/servicios, repositorio, etc.).
- **Gestión de concurrencia y estado:** ¿Cómo maneja este servicio las solicitudes simultáneas, hilos, procesos o almacenamiento de estado en memoria?

3. Catálogo de Endpoints y APIs (REST / HTTP)

Para cada ruta/endpoint expuesto por este servicio, detalla:

- **Ruta y Verbo HTTP** (GET, POST, PUT, PATCH, DELETE).
- **Parámetros de consulta (Query params) y Cabeceras especiales** (ej. tokens de bloqueo, contextos de bypass, etc.).
- **Estructura del Payload (JSON / XML):** Ejemplo de la entrada esperada y del objeto de respuesta exitoso.
- **Códigos de estado HTTP y Manejo de errores:** Describe qué provoca respuestas de error comunes (400 Bad Request, 401 Unauthorized, 409 Conflict, 500 Internal Error) y la estructura del objeto de error devuelto (por ejemplo, el campo `cause` o `validationErrors`).

4. Protocolos en Tiempo Real (WebSockets / SSE)

Si este servicio implementa o interactúa con WebSockets o flujos de eventos (por ejemplo, subprotocolos como `la-core-notifier` o `la-core-alarm`):

- **Endpoints de conexión:** Rutas WebSocket expuestas.
- **Estructura de mensajes:** Formato de los payloads para mensajes recibidos del cliente (ej. `filter\_update`, `ALARM\_READ`) y enviados por el servidor (ej. `IO\_NOTIFY`, `ERRORS\_NOTIFY`, `DB\_ENGINE\_ALARM\_NOTIFY`).
- **Máquina de estados de conexión:** Cómo gestiona el servicio las desconexiones, reconexiones o el envío de pings/keep-alive.

5. Mecanismo de Configuración, Bloqueo y Esquemas

Si este servicio gestiona nodos de configuración (por ejemplo, `config.video.cam`, `config.device.users`, `config.io.inputs`, etc.):

- **Sistema de Bloqueo Pessimistic (Pessimistic Locking):** Explica el ciclo de vida de la adquisición, renovación (`acquireLock`) y liberación (`releaseLock`) del token en el backend. ¿Cómo se valida el parámetro de consulta `token` o el `seed`?
- **Motor de Esquemas y Plantillas:** Lógica detrás de la generación de esquemas de validación (`/properties/schema`) y plantillas de inicialización (`/properties/template`).
- **Fusión de Cambios (Patch Merging):** ¿Cómo procesa y valida el backend las actualizaciones parciales (PATCH) y cómo decide si un cambio requiere reinicio de hardware (`\_rebootRequired`) o emite la cabecera `X-Reboot-Required`?

6. Pipeline de Video, Almacenamiento o Telemetría (Si aplica)

Si este servicio maneja ingesta o streaming de medios:

- **Gestión de flujos de video:** Cómo interactúa con los protocolos de streaming expuestos al cliente (WHEP para WebRTC, HLS indexación `m3u8`, o RTSP/TCP/UDP).
- **Estructura de almacenamiento físico:** Organización y rotación de los datos de video en disco bajo los distintos modos de RAID (disabled, mirror, double-mirror, big, n+1).
- **Métricas de rendimiento:** Qué significan a nivel de ingeniería los datos expuestos en `/video-engine/stats.json` (Message queues, Block pools, latency metrics, etc.).

7. Integración con el Frontend (web-core-view)

- Diseña un diagrama de secuencia lógico o describe el flujo de comunicación típico entre este servicio y la aplicación Angular para las operaciones críticas (ej. Login/Whoami, actualización de firmware, configuración de condiciones I/O, o flujo de renderizado de reportes).

Por favor, sé lo más claro, específico y detallado posible. Evita resúmenes vagos y concéntrate en los contratos de datos reales, algoritmos de validación e interacciones del sistema que me sirvan para redactar mi documentación técnica de fin de grado.

la-core-legacy1

Informe Técnico: Arquitectura y Funcionamiento del Servicio `la-core-legacy`

Este informe documenta detalladamente la arquitectura, los mecanismos de concurrencia, el sistema de enrutamiento REST, el flujo de señales D-Bus, la gestión de bloqueos pesimistas, el almacenamiento RAID y las pautas de integración con el frontend (`web-core-view` en Angular 19) del servicio central de configuración del Grabador de Vídeo en Red (NVR) de Lanaccess.

1. Resumen Ejecutivo y Propósito del Servicio

El servicio `la-core-legacy` actúa como el **núcleo de control y gestor de configuración** dentro del ecosistema del NVR de Lanaccess. Sus responsabilidades principales son:

- * **Gestión Centralizada del Estado y Configuración:** Centraliza la parametrización de dispositivos físicos (cámaras, interfaces de E/S analógicas y digitales, almacenamiento) y parámetros lógicos (puertos, reglas de alarma, proxies, telemetría y usuarios).
- * **Fachada de Interoperabilidad REST/CGI:** Expone una interfaz web HTTP para la administración del NVR, traduciendo peticiones HTTP REST de alto nivel en operaciones de bajo nivel (llamadas a D-Bus, interacción directa con el sistema de archivos del NVR, invocaciones de controladores de hardware, etc.).
- * **Pila Tecnológica:**
 - * **Lenguaje:** C++ moderno compatible con estándares de compilación cruzada mediante CMake para arquitecturas embebidas (`imx8`, `lava`) y PC estándar (`x86`).
 - * **Base de Datos / Persistencia:** Interactúa mediante llamadas de comunicación inter-procesos (IPC) a través del bus de sistema **D-Bus** con el motor de base de datos (`la-db-engine`).
 - * **Servidor Web:** Incorpora un servidor HTTP integrado basado en **libmicrohttpd** para recibir e interpretar las llamadas web.

2. Pila Tecnológica y Modelo de Concurrencia

Una de las particularidades más destacadas del diseño interno de `la-core-legacy` es la emulación de un Sistema Operativo en Tiempo Real (RTOS) sobre un sistema Linux estándar.

Emulación ThreadX sobre POSIX (ThreadX-over-pthread)

Históricamente desarrollado para el RTOS **ThreadX**, el servicio implementa una capa de emulación (`root/core/threadx\_over\_pthread`) que mapea llamadas nativas de ThreadX a llamadas POSIX de Linux (`pthread`):

- * **tx_thread_create:** Crea un hilo del sistema asociándolo internamente a un `pthread`.
- * **TX_QUEUE:** Implementa colas de mensajes inter-hilos protegidas mediante variables de condición y exclusión mutua de POSIX (`pthread\_mutex\_t`).
- * **TX_EVENT_FLAGS_GROUP:** Mapea grupos de banderas de eventos del RTOS (event flags) para la sincronización asíncrona de tareas.

Hilos de Servicio Activos en Arranque

El servicio distribuye sus funciones de procesamiento en múltiples hilos dedicados:

Nombre del Hilo	Función Principal
`svr.WebSupv`	Supervisor del servidor web HTTP, encargado de aceptar conexiones externas a través de `libmicrohttpd`.
`svr.Discovery`	Hilo de descubrimiento de cámaras IP mediante el motor ONVIF/WS4D.
`svr.NTP` / `svr.NTPState`	Gestores de sincronización horaria con servidores NTP externos.
`Telnet Server` / `TelnetSlave`	Terminales de control heredados para depuración del sistema.
`srv.proxy.master`	Enrutador de proxy inverso web para redirección de tráfico de vídeo.
`svr.OS.CMD` / `cli.OS.CMD`	Bucle de ejecución para la terminal de comandos del NVR.
`svr.IfTop` / `svr.Top`	Hilos de recolección de telemetría de rendimiento (CPU, red, memoria).
`drv.Interphone`	Gestor del intercomunicador bidireccional y audio sobre IP.

3. Dispatching y Seguridad de la API REST

El flujo de enrutamiento y ejecución de las llamadas HTTP entrantes está gobernado por un despachador declarativo centralizado.

El Despachador de Rutas (`api\_rest\_dispatch.cpp`)

El despachador recibe la petición analizada por `libmicrohttpd`, extrae los tokens de ruta de la URI y los asocia dinámicamente con un nodo de la estructura jerárquica de la API.

* **Estructura Declarativa (`Entity\_ApiRestElementNodeTree`):** El árbol de endpoints de la API REST se define de forma estática en `api\_rest\_globals.cpp`. Cada nodo representa un recurso (ej. `config`, `device`, `media`, `io`) y registra punteros a funciones de callback para los métodos HTTP soportados (`GET`, `POST`, `PUT`, `DELETE`, `PATCH`).

Autorización y Privilegios

Antes de invocar el manejador registrado para un endpoint, `api\_rest\_dispatch` realiza una validación estricta de privilegios:

1. Obtiene los niveles de acceso del token de sesión HTTP llamando a `cgi\_get\_grant(sock)`.
2. Compara la máscara de permisos del usuario con la máscara de permisos requerida por el nodo de la ruta actual (ej. `USER\_PERMISO\_CONFIGURACION` o `USER\_PERMISO\_CONFIG\_USERS`).
3. Si la validación falla, interrumpe el procesamiento inmediatamente devolviendo un error de tipo `403 Forbidden` (`g\_json\_str\_api\_rest\_service\_error\_privilege`).

4. Notificaciones, D-Bus y Eventos en Tiempo Real

El servicio `la-core-legacy` genera eventos que reflejan cambios de estado del hardware en tiempo real. En lugar de mantener miles de conexiones WebSocket abiertas directamente, el servicio adopta un patrón pub-sub desacoplado: emite señales a través de **D-Bus** en la ruta de objeto `/com/lanaccess/sysevents` y la interfaz `com.lanaccess.sysevents`. Posteriormente, una pasarela independiente (como el servicio `la-core-notifier`) escucha estas señales D-Bus y las transmite a los clientes frontend mediante WebSockets.

Principales Señales D-Bus y sus Estructuras de Datos

1. `IO\_NOTIFY` (Firma D-Bus: `a(sssss)`)

Se emite de forma masiva o individualizada cuando cambia el estado de un componente de entrada/salida. El payload es un array de tuplas de 5 elementos en formato string:

- * `strTopic`: Canal o tipo de evento (ej. `"io/input"`, `"io/virtual_input"`, `"io/camera/input"`, `"io/output"`, `"io/camera/status"`, `"io/camera/motion"`).
- * `strTime`: Marca de tiempo formateada en formato UTC ISO 8601 (ej. `"2026-06-10T10:45:00Z"`).
- * `strId`: Identificador físico de la entrada o salida (ej. `"1"`).
- * `strIdr`: Identificador de contexto relativo (ej. ID de cámara).
- * `strData`: Estado actual textual del componente:
 - * **Entradas**: `"idle"`, `"on"`, `"open-circuit"`, `"short-circuit"`, `"disabled"`, `"disarmed"`.
 - * **Salidas**: `"activated"`, `"deactivated"`, `"not-configured"`.
 - * **Cámaras**: `"no-signal"`, `"watching"`, `"recording"`, `"pre-recording"`, `"recording-alarm"`, `"idle"`.

2. `ALARM\_NOTIFY` (Firma D-Bus: `ssssssss`)

Emite notificaciones cuando una regla de alarma se activa en el sistema. Los parámetros son posicionales:

1. `zulu_datetime`: Fecha/hora UTC de la alarma.
2. `code`: Código de la alarma.
3. `status`: Estado (`"active"`, `"restored"`, etc.).
4. `code_id`: Identificador único de instancia.
5. `description`: Descripción textual de la alarma.
6. `priority`: Nivel de severidad o prioridad.
7. `alarm_type`: Tipo lógico de la alarma.
8. `camera_num`: Cámara asociada a la que hace referencia el evento.

3. `ERRORS\_NOTIFY` (Firma D-Bus: `ii`)

Reporta eventos globales de salud o fallos en el hardware del NVR:

- * `num_errors` (int): Cantidad total de errores activos registrados en el sistema.
- * `pending_errors` (int): Cantidad de errores que no han sido reconocidos/vistos por un administrador.

5. Gestión de Configuración y Pessimistic Locking

Para salvaguardar la integridad de las modificaciones en un entorno multiusuario, `la-core-legacy` implementa una estrategia de **bloqueo pesimista (Pessimistic Locking)** para los recursos de configuración del NVR.

```
sequenceDiagram
    autonumber
    actor FE as Frontend (Angular)
    participant BE as la-core-legacy (REST)
    participant DB as la-db-engine (D-Bus)

    FE->>BE: GET /properties/lock?node=config.network (seed)
    BE->>BE: Comprueba colisión de Locks en jerarquía
    BE-->>FE: HTTP 200 { "token": "32_char_token", "expires": 30 }
    Note over FE,BE: El cliente adquiere exclusión de escritura

    FE->>BE: PATCH /properties/data?node=config.network (JSON Patch)
    Note over BE: Valida Token + IP + User en sesión
    BE->>BE: Convierte JSON a Flat Map
    BE->>BE: Valida tipos remotos con sys_prop_validator
    BE->>BE: Evalúa check_if_reboot_needed_smart()
    BE->>DB: DBUS: set_list_of_properties(strList)
    BE-->>FE: HTTP 200 [Datos Actualizados] + (Headers: X-Reboot-Required)

    FE->>BE: POST /properties/unlock?node=config.network (token)
    BE->>BE: Elimina Lock de g_active_locks
    BE-->>FE: HTTP 200 { "result": "ok" }
```

El Ciclo de Vida del Bloqueo

1. **Adquisición (`lock`):** Un cliente solicita `/properties/lock` indicando el `node` que desea modificar y un valor de protección `seed`. El servidor crea una entrada en la estructura global `g\_active\_locks` protegida por un mutex, asocia el usuario y la IP, y devuelve un `token` único de 32 caracteres y un tiempo de expiración (`LOCK\_DURATION\_SECONDS`, por defecto 30 segundos). El servidor detecta colisiones tanto si el nodo ya está bloqueado, como si existe un bloqueo en un nodo superior o inferior de la jerarquía.
2. **Validación de Escritura:** En cualquier solicitud de modificación (`PATCH`, `PUT`), el servidor ejecuta `check\_lock` para verificar que el usuario actual es dueño del bloqueo sobre el nodo de destino utilizando el token aportado.
3. **Liberación (`unlock`):** El cliente realiza un POST a `/properties/unlock` aportando el token para liberar el recurso de inmediato. Alternativamente, un hilo en segundo plano limpia periódicamente los bloqueos cuya marca de tiempo ha expirado.

Lógica de Patch Masivo (RFC 6902 JSON Patch)

Cuando una petición de parcheado se realiza con el Content-Type `application/json-patch+json`, el backend invoca `api\_rest\_device\_service\_properties\_data\_patch\_rfc6902`:

- * ****Expansión de Comodines (`{}`):**** Si el campo `path` incluye comodines (ej. `config.cameras.{}.enabled`), la función busca recursivamente los subnodos instanciados en la base de datos (ej. `001`, `002`) y clona la operación para cada uno de ellos.
- * ****Filtros Posicionales (`where`):**** Permite filtrar los comodines mediante expresiones condicionales (ej. `"where": "ip\_camera == true"`), evaluando propiedades locales del elemento antes de aplicar la modificación.
- * ****Aplanamiento de JSON (Flat Maps):**** El JSON jerárquico modificado es aplanado en un mapa clave-valor plano compatible con el modelo de propiedades jerárquicas del grabador.

Validación Estructural y D-Bus

El mapa plano a escribir es validado dinámicamente llamando al servicio local `sys\_prop\_validator` (`remote\_validate\_list`), el cual lee los esquemas JSON de definición generados en tiempo real por el motor de base de datos. Si la validación es correcta, los cambios son escritos atómicamente llamando a `dbus\_querier\_call\_set\_list\_of\_properties` para que se apliquen en `la-db-engine`.

Detección Inteligente de Reinicios Necesarios (`check\_if\_reboot\_needed\_smart`)

El backend decide si una modificación de configuración requiere un reinicio de hardware mediante las siguientes reglas:

- * Itera sobre cada propiedad modificada y localiza su definición en el esquema JSON (`find\_property\_schema`).
- * Verifica si el esquema contiene la propiedad `"\_rebootRequired": true`.
- * Compara el nuevo valor entrante con el valor persistido actual. Si y solo si el valor difiere, la función devuelve `true`.
- * Si se trata de una operación `PUT`, comprueba si propiedades con el flag `\_rebootRequired` han sido omitidas de la petición (lo cual implica que retornarán a su valor por defecto de fábrica, considerándose una modificación crítica).
- * En caso de confirmarse la necesidad de reinicio, invoca a `system\_mark\_reboot\_required()`, lo que establece la bandera global `global\_reboot\_required = true`. En la respuesta HTTP subsiguiente, el servidor añade la cabecera HTTP `X-Reboot-Required: true`.

6. Pipeline de Vídeo, Almacenamiento o Telemetría

El servicio expone estadísticas dinámicas del hardware y controla los arreglos de discos físicos del grabador.

Estadísticas de Receptores de Vídeo

A través del endpoint `/device-service/reports/video/stats`, el backend invoca a `video\_engine\_call\_get\_receivers\_report()`. Esta llamada interna interactúa con el motor de streaming del grabador y obtiene un reporte tabulado que el servicio parsea dinámicamente a JSON para exponerlo en la API:

- * Identificación de receptores (`receiver` / `track`).
- * Estado de la conexión del flujo (`Connected`, `Connecting`, `NoConnected`).
- * Ancho de banda consumido en kbps (`bw`).

- * Paquetes totales recibidos (`rx\_pkts`).
- * Métricas de calidad de flujo: fluctuación de retardo (`jitter`) y pérdida acumulada de paquetes (`lost`).

Almacenamiento y Modos RAID Proprietarios

El módulo de disco duro (`THERE\_IS\_HARDDISK\_MODULE`) gestiona directamente la configuración de almacenamiento del NVR, soportando múltiples topologías RAID:

Modo RAID	Código	Descripción del Sistema
HD_RAID_MODE_DISABLED	99	Modo de disco único (Single). Sin tolerancia a fallos.
HD_RAID_MODE_0_BIG	0	Modo Agregado (Aggregate/Big). Suma la capacidad de todos los discos.
HD_RAID_MODE_1_MIRROR	1	Espejo simple (Mirror). Réplica idéntica entre el disco 1 y 2.
HD_RAID_MODE_DOUBLE_MIRROR	2	Doble Espejo. Dos conjuntos independientes de discos en espejo.
HD_RAID_MODE_DINAMICO	3	RAID Dinámico en red (RAID LAN). Almacenamiento replicado de forma activa/pasiva a través de múltiples NVRs Lanaccess de la red local.

El estado del almacenamiento y del RAID es serializado en JSON mediante `hd\_get\_configured\_disks\_json()`, `hd\_get\_detected\_disks\_json()`, y `hd\_get\_raid\_state\_json()`, reportando datos sobre el disco (modelo, número de serie, tamaño en GB, fecha de instalación en formato `MM/AAAA` extraída por `system\_info\_get\_disk\_date`) y detalles del estado de reconstrucción del espejo (`REBUILDING\_1\_TO\_2`, `OK\_1\_2\_NEW`, etc.).

La operación de formateo se desencadena invocando el endpoint correspondiente que ejecuta `hd\_format\_disks()`, determinando según el modo RAID si se debe borrar un disco individual (`hd\_erase\_one\_disk\_format`) o la totalidad del conjunto (`hd\_erase\_all\_disks\_format`).

7. Integración con el Frontend `web-core-view` (Angular 19)

Para que el frontend de control interactúe correctamente con el backend `la-core-legacy`, se sugieren las siguientes pautas de diseño:

1. **Ciclo de Formulario Reactivo y Locking:**

- * Al abrir un formulario de edición para cualquier nodo de configuración, la aplicación Angular debe enviar primero una solicitud GET a `/properties/lock?node=ruta.nodo` para reclamar el bloqueo.

- * Mientras el bloqueo esté activo, debe iniciarse un temporizador en el cliente para monitorizar la caducidad (30 segundos). Si el usuario continúa interactuando con el formulario, el cliente puede enviar solicitudes periódicas para renovar el bloqueo.

- * Al cerrar o destruir el componente del formulario, debe lanzarse un POST a ``/properties/unlock`` para liberar el nodo para otros administradores.
2. ****Operaciones PATCH Masivas y Eficientes:****
- * Al guardar los cambios del formulario, es altamente recomendado utilizar el tipo de contenido ``application/json-patch+json`` con peticiones de tipo RFC 6902.
 - * Esto permite empaquetar en un único mensaje REST múltiples adiciones y reemplazos, reduciendo sustancialmente el número de transacciones sobre la red y sobre la base de datos D-Bus interna del grabador.
3. ****Monitoreo del Estado de Reinicio:****
- * El frontend debe configurar un interceptor HTTP en Angular que revise la presencia de la cabecera ``X-Reboot-Required`` en las respuestas de la API de propiedades.
 - * Si el valor retornado es ``true``, la interfaz debe alertar visualmente al usuario indicando que los cambios guardados se aplicarán por completo solo después de reiniciar el dispositivo.
4. ****Consumo de Eventos en Tiempo Real:****
- * Para renderizar alarmas u monitorizar el estado de las entradas analógicas y digitales en tiempo real (por ejemplo, para cambiar los colores de los iconos de los sensores en el plano del NVR), Angular debe suscribirse al servicio WebSocket central expuesto por la pasarela ``la-core-notifier``.
 - * La interfaz debe mapear las notificaciones recibidas con el formato ``IO_NOTIFY`` y decodificar el valor textual de ``strData`` para actualizar reactivamente la vista de estado de las entradas e interfaces del sistema.

la-core-legacy2

Informe Técnico: Arquitectura y Funcionamiento del Servicio `la-core-legacy`

Este informe documenta detalladamente la arquitectura, los mecanismos de concurrencia, el flujo criptográfico de inicio de sesión, la jerarquía de permisos (grants), el sistema de enrutamiento REST, el flujo de señales D-Bus, la gestión de bloqueos pesimistas, el almacenamiento RAID, los asistentes de configuración paso a paso (PCFG/Wizard) y las pautas de integración con el frontend (`web-core-view` en Angular 19) del servicio central del Grabador de Vídeo en Red (NVR) de Lanaccess.

1. Resumen Ejecutivo y Propósito del Servicio

El servicio `la-core-legacy` actúa como el **núcleo de control y gestor de configuración** dentro del ecosistema del NVR de Lanaccess. Sus responsabilidades principales son:

- * **Gestión Centralizada del Estado y Configuración:** Centraliza la parametrización de dispositivos físicos (cámaras, interfaces de E/S analógicas y digitales, almacenamiento) y parámetros lógicos (puertos, reglas de alarma, proxies, telemetría y usuarios).
- * **Fachada de Interoperabilidad REST/CGI:** Expone una interfaz web HTTP para la administración del NVR, traduciendo peticiones HTTP REST de alto nivel en operaciones de bajo nivel (llamadas a D-Bus, interacción directa con el sistema de archivos del NVR, invocaciones de controladores de hardware, etc.).
- * **Pila Tecnológica:**
 - * **Lenguaje:** C++ moderno compatible con estándares de compilación cruzada mediante CMake para arquitecturas embebidas (`imx8`, `lava`) y PC estándar (`x86`).
 - * **Base de Datos / Persistencia:** Interactúa mediante llamadas de comunicación inter-procesos (IPC) a través del bus de sistema **D-Bus** con el motor de base de datos (`la-db-engine`).
 - * **Servidor Web:** Incorpora un servidor HTTP integrado basado en **libmicrohttpd** para recibir e interpretar las llamadas web.

2. Pila Tecnológica y Modelo de Concurrencia

Una de las particularidades más destacadas del diseño interno de `la-core-legacy` es la emulación de un Sistema Operativo en Tiempo Real (RTOS) sobre un sistema Linux estándar.

Emulación ThreadX sobre POSIX (ThreadX-over-pthread)

Históricamente desarrollado para el RTOS **ThreadX**, el servicio implementa una capa de emulación (`root/core/threadx\_over\_pthread`) que mapea llamadas nativas de ThreadX a llamadas POSIX de Linux (`pthread`):

- * `tx_thread_create`: Crea un hilo del sistema asociándolo internamente a un `pthread`.
- * `TX_QUEUE`: Implementa colas de mensajes inter-hilos protegidas mediante variables de condición y exclusión mutua de POSIX (`pthread\_mutex\_t`).

* ``TX_EVENT_FLAGS_GROUP``: Mapea grupos de banderas de eventos del RTOS (event flags) para la sincronización asíncrona de tareas.

Hilos de Servicio Activos en Arranque

El servicio distribuye sus funciones de procesamiento en múltiples hilos dedicados:

Nombre del Hilo	Función Principal
<code>`svr.WebSupv`</code>	Supervisor del servidor web HTTP, encargado de aceptar conexiones externas a través de <code>`libmicrohttpd`</code> .
<code>`svr.Discovery`</code>	Hilo de descubrimiento de cámaras IP mediante el motor ONVIF/WS4D.
<code>`svr.NTP`</code> / <code>`svr.NTPState`</code>	Gestores de sincronización horaria con servidores NTP externos.
<code>`Telnet Server`</code> / <code>`TelnetSlave`</code>	Terminales de control heredados para depuración del sistema.
<code>`srv.proxy.master`</code>	Enrutador de proxy inverso web para redirección de tráfico de vídeo.
<code>`svr.OS.CMD`</code> / <code>`cli.OS.CMD`</code>	Bucle de ejecución para la terminal de comandos del NVR.
<code>`svr.IfTop`</code> / <code>`svr.Top`</code>	Hilos de recolección de telemetría de rendimiento (CPU, red, memoria).
<code>`drv.Interphone`</code>	Gestor del intercomunicador bidireccional y audio sobre IP.

3. Autenticación (Login) y Gestión de Sesiones Criptográficas

Para asegurar que las credenciales de administración no viajen en texto plano ni sean vulnerables a ataques de hombre en el medio (MITM) o de replay (reproducción), ``la-core-legacy`` implementa una arquitectura de **criptografía híbrida asimétrica/simétrica (RSA + AES-128)**.

```mermaid

sequenceDiagram

autonumber

actor FE as Frontend (Angular)

participant BE as la-core-legacy (RPC)

Note over FE: Genera par RSA temporal<br/>(PublicKeyN, PublicKeyE)

FE->>BE: POST /rpc/GetEncryptedAES16Key { PublicKeyN, PublicKeyE }

Note over BE: Genera Session ID y<br/>Clave AES-128 aleatoria

BE->>BE: Cifra Clave AES con RSA del cliente

BE-->>FE: HTTP 200 { GetEncryptedAES16KeyResponse: Hex(AES\_Encrypted) } +

Cookie SessionID

Note over FE: Decodifica Clave AES con su RSA privada

Note over FE: Cifra credenciales con Clave AES<br/>(User + Password)

FE->>BE: POST /rpc/Login { User, EncryptedPassword (Base64) } + Cookie SessionID

BE->>BE: Recupera Clave AES de la Sesión

BE->>BE: Descifra credenciales con AES-128-CBC

BE->>BE: Verifica credenciales en base de datos

BE-->>FE: HTTP 200 { LoginResponse: "success" } o "locked" / "unauthorized"

...

#### ### Flujo de Negociación y Cifrado

##### 1. \*\*Negociación de Clave de Sesión (`GetEncryptedAES16Key`):\*\*

- \* El frontend genera un par de claves RSA en el navegador y envía la clave pública (Módulo `PublicKeyN` y Exponente `PublicKeyE`) mediante una solicitud POST a `/rpc/GetEncryptedAES16Key`.

- \* El backend detecta si existe una sesión activa (mediante la cookie `SessionId`). Si no existe, invoca `web\_login\_prepare\_new\_session` para inicializar el identificador y genera una \*\*clave simétrica AES-128 temporal\*\*.

- \* Utilizando OpenSSL (`newRSA`), el backend cifra la clave simétrica AES generada usando la clave pública RSA provista por el frontend.

- \* La clave cifrada se codifica en formato hexadecimal y se devuelve en el cuerpo JSON: `{"GetEncryptedAES16KeyResponse": "<hex\_data>"}`.

##### 2. \*\*Transmisión de Credenciales Cifradas (`Login`):\*\*

- \* El cliente descifra la clave simétrica con su clave privada RSA, obteniendo la clave AES de sesión.

- \* El cliente cifra los campos de credenciales (`User`, `Password`) con esta clave AES de sesión.

- \* El cliente envía la contraseña cifrada simétricamente codificada en Base64 en el parámetro `EncryptedPassword` a `/rpc/Login`.

- \* El backend recupera la clave AES correspondiente al `SessionId`, decodifica el Base64, extrae el Vector de Inicialización (IV) de los primeros 16 bytes, realiza el descifrado simétrico AES-CBC y recupera las credenciales del usuario.

##### 3. \*\*Protección contra Ataques de Fuerza Bruta y Replay:\*\*

- \* El backend invoca a `replay\_attack\_is\_active()` y `replay\_attack\_process\_unauthorized\_auth()` en caso de fallos.

- \* Si una IP supera el límite de intentos fallidos permitidos, se bloquea la autenticación temporalmente, respondiendo con `{"LoginResponse": "locked"}`.

- \* Si el login es correcto, se registra el estado mediante `web\_login\_set\_session\_id\_success\_as\_form` y se responde con `{"LoginResponse": "success"}`.

#### ### Gestión de Permisos y Niveles de Acceso (Grants)

Las credenciales verificadas determinan una máscara de bits de privilegios (Grants) almacenada en la sesión. Estos niveles de acceso están definidos en `command\_defines.h`:

- \* \*\*`USER\_PERMISO\_GESTION` (`0x001`):\*\* Permisos de administración global del sistema (reinicios, formateos, etc.).

- \* \*\*`USER\_PERMISO\_CONFIGURACION` (`0x002`):\*\* Permisos para cambiar propiedades y configuraciones del NVR.

- \* \*\*`USER\_PERMISO\_VIVO` (`0x004`) y `USER\_PERMISO\_VIVO\_LIMITADO` (`0x100`):\*\* Permisos para la visualización del flujo de vídeo en directo.

- \* \*\*`USER\_PERMISO\_GRABACIONES` (`0x008`):\*\* Permisos para reproducir vídeo almacenado.

- \* `USER_PERMISO_DESCARGAS` (`0x010`):` Permisos para exportar/descargar fragmentos de vídeo.
- \* `USER_PERMISO_ASISTENTE` (`0x020`)` y `USER_PERMISO_ASISTENTE_PLUS` (`0x040`):` Permisos especiales de configuración simplificada (wizards).
- \* `USER_PERMISO_UPDATE_FW` (`0x080`):` Permisos para actualización de firmware.
- \* `USER_PERMISO_CONFIG_USERS` (`0x200`):` Permiso específico para crear y modificar cuentas de usuario y perfiles de seguridad.

---

## ## 4. Estructura de Endpoints de la API REST

El enrutamiento está gobernado por el árbol jerárquico estático declarado en `api_rest_globals.cpp``. A continuación se detallan los endpoints agrupados por su responsabilidad de servicio:

### ### 1. Gestión de Propiedades y Bloqueos

Ubicados bajo la ruta `/device-service/properties/...``. Requieren el grant `USER_PERMISO_CONFIGURACION`` para la integridad de datos.

- \* `/schema` (GET):` Obtiene los esquemas de propiedades JSON generados dinámicamente por la base de datos para validar los campos del sistema.
- \* `/data` (GET, POST, PUT, DELETE, PATCH):` Endpoint principal para lectura y manipulación de las propiedades de configuración. Acepta parches RFC 6902.
- \* `/template` (GET):` Provee plantillas preconfiguradas de propiedades para simplificar la creación de recursos.
- \* `/lock` (GET):` Reclama el bloqueo pesimista de un nodo de propiedades. Devuelve un token UUID de 32 caracteres.
- \* `/unlock` (GET):` Libera el bloqueo pesimista entregando el token.
- \* `/locklist` (GET):` Devuelve una lista de los nodos bloqueados actualmente por IP y usuario.

### ### 2. Recursos y Configuración Física

Ubicados bajo `/device-service/resources/...``. Requieren permisos de tipo `USER_PERMISO_TODOS`` (para consultas generales de sólo lectura en componentes inofensivos) o `USER_PERMISO_CONFIGURACION`` / `USER_PERMISO_CONFIG_USERS`` (para manipulación).

- \* `/users` y /users/map` / /users/map/profiles` (GET):` Consulta la lista de usuarios y los perfiles de seguridad configurados.
- \* `/clock`, /clock/timezones` (GET) y /clock/datetime` (GET, PUT):` Sincronización y ajuste de fecha/hora y zona horaria.
- \* `/video` y /video/cameras` (GET):` Catálogo de cámaras físicas y sus índices en el grabador.
- \* `/video/ipcameras` (GET):` Listado de cámaras IP y sus configuraciones detalladas de red/streaming.

- \* ``/video/ipcameras/video_encoder_configuration_options`` (GET) y ``/video/ipcameras/audio_encoder_configuration_options`` (GET): Opciones del codificador multimedia por ONVIF.
- \* ``/video/ipcameras/lock`` y ``/video/ipcameras/unlock`` (GET, PUT): Bloqueo específico del recurso de cámara IP.
- \* ``/hd`` (GET): Listado de almacenamiento detectado y configurado en el sistema.
- \* ``/hd/format`` (PUT): Inicia el formateo de discos duros según la topología RAID configurada.
- \* ``/io-space`` y ``/io-space/dim`` (GET): Dimensiones físicas de los bloques de E/S analógicos y digitales.
- \* ``/io-space/inputs/physical`` / ``/virtual`` / ``/lava`` / ``/iv`` / ``/ios`` (GET): Índices de entradas físicas, virtuales, LAVA, analítica o clientes IOS instaladas.
- \* ``/io-space/outputs/commute`` (GET): Permite conmutar (abrir/cerrar) un canal de salida digital del NVR.

### ### 3. Proxy de Vídeo

- \* ``/device-service/proxy/capabilities`` (GET): Capacidades del proxy inverso integrado.
- \* ``/device-service/proxy/rules`` (GET, POST): Listado y adición de reglas de proxy.
- \* ``/device-service/proxy/rules/{id}`` (PUT, DELETE): Modificación o eliminación de una regla de proxy por su identificador.

### ### 4. Telemetría, Diagnósticos y Reportes

Rutas de sólo lectura situadas en ``/device-service/reports/...``.

- \* ``/system_load`` / ``/system_load/cpu`` / ``/system_load/network`` (GET): Telemetría de uso del sistema.
- \* ``/video/stats`` (GET): Estadísticas dinámicas de red de receptores de streaming (paquetes perdidos, jitter, ancho de banda).
- \* ``/video/detail`` y ``/video/flux`` (GET): Detalles técnicos de los flujos de vídeo activos.
- \* ``/ip_cameras/general`` (GET) y ``/ip_cameras/ops`` (GET, POST): Logs operacionales del motor de descubrimiento ONVIF.
- \* ``/reset``, ``/reset/core_dumps``, ``/reset/reset_list`` (GET): Registros de caídas del kernel, volcados de memoria e historial de reinicios físicos.
- \* ``/io-space/physical_inputs`` / ``/outputs`` (GET): Estados activos de sensores y actuadores físicos del dispositivo.

---

## ## 5. Gestión del Configuración y Bloqueo Pesimista (Pessimistic Locking)

En sistemas embebidos de seguridad donde múltiples usuarios pueden intentar modificar el grabador a la vez, se requiere evitar la condición de carrera sobre la configuración física. ``la-core-legacy`` resuelve esto con un sistema de **bloqueo pesimista**.

### ### Adquisición y Mutex de Bloqueo

1. **Solicitud de Lock:** Un cliente realiza una petición ``GET /device-service/properties/lock?node=config.cameras.001&seed=12345``.



2. **Resolución de Jerarquía:** El backend bloquea con un mutex interno su tabla de locks ``g_active_locks``. Valida que el nodo solicitado no esté bloqueado. Si un nodo padre (ej. ``config``) o un nodo hijo (ej. ``config.cameras.001.enabled``) ya está bloqueado, se produce una colisión de jerarquía y devuelve un fallo.
3. **Generación de Token:** Si no hay colisiones, se genera un token UUID y se almacena junto al identificador de usuario, la IP origen, y una marca de expiración de 30 segundos (``LOCK_DURATION_SECONDS``). El token se le retorna al cliente.
4. **Uso del Token:** En cada petición ``PATCH /properties/data`` o ``PUT /properties/data``, el cliente debe adjuntar la cabecera o el parámetro ``token``. El backend invoca a ``check_lock`` para verificar la propiedad del token antes de autorizar la escritura.
5. **Liberación o Caducidad:** El cliente libera explícitamente mediante ``/unlock``. Si el cliente pierde la conexión o no completa el formulario, un hilo de limpieza periódico de ``la-core-legacy`` detecta que el token ha superado los 30 segundos y lo elimina de la estructura, permitiendo a otros administradores bloquear la sección.

---

## ## 6. Sistema de Asistentes de Configuración (Wizards y PCFG)

El sistema soporta asistentes interactivos paso a paso para simplificar la puesta en marcha inicial del grabador o de módulos complejos. Existen dos vertientes dentro de la base de código heredada:

### ### 1. Progressive Configuration (PCFG)

El módulo ``pcfg`` (``root/webinteface/pcfg/``) implementa un configurador secuencial o progresivo gestionado por estados en memoria antes de persistir los datos finales.

- \* **Inicialización:** El asistente se activa accediendo a ``pcfg_init.html`` que ejecuta ``wizard2_cgi_enter_pcfg``. Esto reinicia la variable de estado global ``pcfgVariables`` y pone en cero el acceso.

- \* **Estructura de Pasos:** Los ficheros se dividen por áreas funcionales y pasos:

- \* **Vídeo e IP:** Controlado por pasos como ``PCFG_VIDEO_STEP_IP_SETTINGS``, ``PCFG_VIDEO_STEP_IP_SETTINGS_FOCUS`` y ``PCFG_VIDEO_STEP_IP_PRIVACY_MASK``. El backend ejecuta ``wz2_ipCam_step.cpp`` y refresca el estado en red vía ``wz2_ipCam_refresh.cpp`` (``pcfg_video_refresh_ip_settings``).

- \* **Usuarios:** Pasos en ``pcfg_general_step_users.cpp`` para definir claves y accesos iniciales.

- \* **Fechas y Reloj:** ``pcfg_clock_process.cpp`` y ``pcfg_clock_default.cpp``.

- \* **Horarios y Reglas de Eventos:** Pasos intermedios administrados por ``pcfg_crear_elemento_horario.cpp`` y ``pcfg_step_event_horarios.cpp``.

- \* **Flujo Transaccional:** A medida que el usuario avanza por los pasos, los datos intermedios se guardan en la estructura temporal ``pcfgVariables``. Al llegar al paso final, el asistente ejecuta ``pcfg_save_changes.cpp`` para validar globalmente la estructura y volcarla de forma atómica a la base de datos D-Bus del NVR.

### ### 2. CGI Wizard (Wizard2)

El componente ``wizard2`` (``root/core/wizard2/``) expone los asistentes tradicionales renderizados por el servidor web legacy:

- \* Registra las páginas base `wizard2\_index.html`, `wizard2\_init.html`, `wizard2\_main.html` asociándolas a los grants `USER\_PERMISO\_ASISTENTE` o `USER\_PERMISO\_ASISTENTE\_PLUS`.
- \* Maneja controles interactivos como controles deslizantes (`wizard2\_slider.js`), validaciones del lado del cliente (`wizard2\_validation.js`) y previsualización de vídeo integrado (`wizard2\_video.js`).
- \* Permite generar reportes XML al final del flujo del asistente (`wizard2\_report.xml` mediante `wizard2\_cgi\_report\_xml.cpp`).

---

## ## 7. Integración con el Frontend `web-core-view` (Angular 19)

Para implementar el panel de administración en Angular 19 interactuando con la API y los asistentes del grabador, sigue estas pautas técnicas:

1. **\*\*Negociación Criptográfica en el Guard de Login:\*\***
  - \* Antes de solicitar el formulario de credenciales, el cliente Angular debe generar un par de claves RSA (usando Web Crypto API o librerías optimizadas).
  - \* Envía la clave pública a `/rpc/GetEncryptedAES16Key` y guarda la clave AES-128 simétrica devuelta en su almacenamiento temporal de sesión.
  - \* Cifra el string del password con la clave AES y envía la solicitud POST final a `/rpc/Login`.
2. **\*\*Control Reactivo de Bloqueos:\*\***
  - \* Utiliza un servicio global en Angular (`LockManagerService`) que llame a `/properties/lock` al activar rutas de edición de formularios.
  - \* Implementa un `interval` que refresque el bloqueo de forma automática cada 20 segundos para evitar que expire a los 30 segundos mientras el usuario edita el formulario.
  - \* Utiliza el ciclo de vida `ngOnDestroy` del componente o un Guard de tipo `CanDeactivate` para invocar a `/properties/unlock` cuando el usuario cambie de pantalla o cancele la edición.
3. **\*\*Encapsulación de Pasos (Wizards) en Angular:\*\***
  - \* El asistente del frontend no necesita interactuar con el flujo HTML legacy de `wizard2`. En su lugar, el frontend puede recrear los componentes paso a paso (Configuración de Red -> Creación de Usuarios -> Detección de Cámaras).
  - \* Para persistir las configuraciones temporales de forma progresiva, Angular puede ir armando la estructura JSON de propiedades y enviarla mediante peticiones `PATCH /properties/data` al nodo bloqueado, utilizando el validador estructural de la API para avisar de errores de datos paso por paso.
4. **\*\*Consumo de Notificaciones por WebSocket:\*\***
  - \* Los sensores, cámaras y alarmas mostrados en el salpicadero principal de Angular 19 deben suscribirse al canal WebSocket provisto por la pasarela de eventos del NVR.
  - \* Al recibir mensajes bajo el tópico `IO\_NOTIFY`, el frontend debe parsear los parámetros y mapear dinámicamente los valores del string `strData` (`"idle"`, `"watching"`, `"on"`) para reflejar los estados en los widgets de la interfaz.

la-core-legacy3

## # Informe Técnico Completo: Arquitectura Interna del Servicio `la-core-legacy`

Este informe contiene una documentación técnica exhaustiva del servicio `la-core-legacy`, componente central de control, configuración e interoperabilidad de la plataforma de Grabación de Vídeo en Red (NVR) de Lanaccess. Su propósito es servir como material directo y base teórica/práctica para la redacción del Trabajo de Fin de Grado (TFG) de Ingeniería Informática, detallando la interacción entre el backend embebido en C++ y el nuevo frontend web desarrollado en Angular 19 (`web-core-view`).

---

### ## ÍNDICE DE CAPÍTULOS

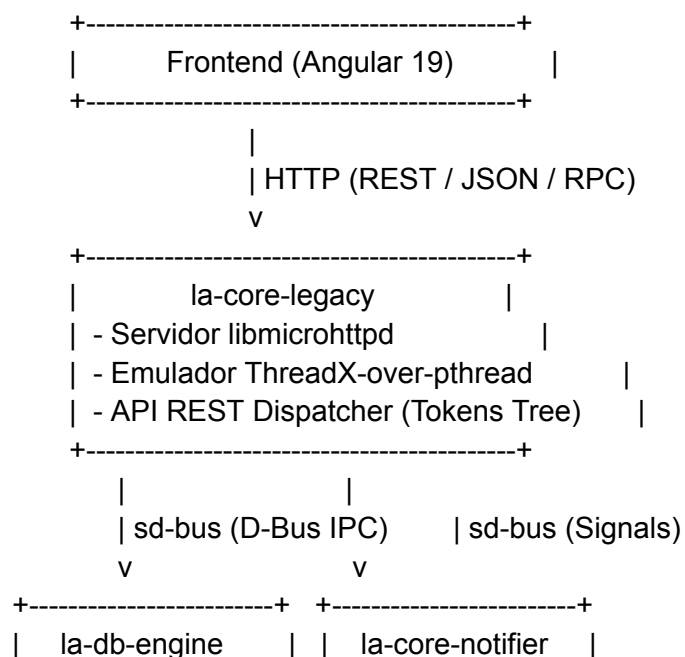
1. **Arquitectura General del Sistema e Interoperabilidad**
2. **Modelo de Concurrencia: Emulación RTOS ThreadX sobre POSIX**
3. **Inicio de Sesión (Login) y Criptografía Híbrida**
4. **Modelo de Seguridad y Roles (Grants)**
5. **Jerarquía REST y Catálogo Detallado de Endpoints**
6. **Gestión de Configuración y Bloqueo Pesimista (Pessimistic Locking)**
7. **El Motor de Asistentes de Configuración Progresiva (PCFG y Wizard2)**
8. **Pipeline de Hardware, RAID, Telemetría y Watchdog del Sistema**
9. **Guía de Integración Práctica con el Frontend Angular 19**

---

### ## 1. Arquitectura General del Sistema e Interoperabilidad

El servicio `la-core-legacy` actúa como la fachada de control del grabador. Su rol principal es recibir las solicitudes HTTP desde el exterior e interactuar con los diferentes demonios internos mediante el bus del sistema Linux.

...



| (Propiedades y Config) | | (WebSocket Gateway) |

+-----+ +-----+

...

#### ### Relación con el Sistema Operativo y Demonios

El NVR opera sobre una distribución Linux embebida a medida (compilada cruzada para procesadores ARM i.MX8 y arquitecturas de PC x86). `la-core-legacy` no implementa persistencia local directa para la configuración en ficheros de texto plano o bases de datos SQL relacionales, sino que adopta una arquitectura desacoplada orientada a servicios:

\* **Motor de Base de Datos (`la-db-engine`):** Es el demonio responsable de almacenar, validar y persistir las propiedades del grabador. `la-core-legacy` se comunica con él enviando y solicitando propiedades en formato clave-valor plano a través de llamadas de D-Bus en la interfaz `com.lanaccess.properties`.

\* **Gestión de Hardware Directa:** Para interactuar con aspectos físicos que escapan al control de la base de datos (por ejemplo, el control de los LEDs frontales de la carcasa, la conmutación de salidas de relés, la lectura directa de sensores o la monitorización del hardware RAID), el servicio invoca APIs de bajo nivel que disparan llamadas a `com.lanaccess.gpiomanager` y `com.lanaccess.licenser`.

\* **Sistema de Notificaciones (`la-core-notifier`):** Es una pasarela WebSocket independiente. Cuando ocurre un evento de hardware (por ejemplo, pérdida de señal en una cámara o activación de un sensor), `la-core-legacy` emite una señal de tipo "Broadcast" en D-Bus. `la-core-notifier` captura la señal y la retransmite a través de WebSockets directamente al frontend Angular 19.

---

## ## 2. Modelo de Concurrencia: Emulación RTOS ThreadX sobre POSIX

Una de las características más singulares del sistema es que fue diseñado originalmente para ejecutarse sobre el Sistema Operativo en Tiempo Real (RTOS) **ThreadX** de Express Logic. Al migrar el software a plataformas Linux Linux estándar, se implementó un wrapper denominado **ThreadX-over-pthread** (`root/core/threadx_over_pthread/`).

#### ### Arquitectura de la Capa de Emulación

Esta capa traduce los tipos primitivos del RTOS a llamadas nativas del estándar POSIX:

\* **Hilos (`TX\_THREAD`):** Se mapean a estructuras del sistema de hilos de Linux (`pthread_t`). Cada hilo emulado tiene su prioridad dinámica, su tamaño de pila y un bucle que monitoriza la variable `b_exit` para garantizar paradas seguras.

\* **Colas de Mensajes (`TX\_QUEUE`):** Se simulan utilizando un buffer circular protegido por un Mutex (`pthread_mutex_t`) y una variable de condición (`pthread_cond_t`). El hilo productor encola un mensaje y despierta al hilo consumidor bloqueado en espera activa.

\* **Grupos de Eventos (`TX\_EVENT\_FLAGS\_GROUP`):** Son máscaras de bits binarias que permiten la sincronización de múltiples hilos de forma no bloqueante. Se emulan utilizando semáforos y bloqueos de exclusión mutua para leer y escribir estados de flags atómicamente.

#### #### Hilos Críticos en la API REST

Al arrancar el servicio, se inicializan diversos hilos con responsabilidades y prioridades bien delimitadas:

1. `svr.WebSupv`` (Prioridad Alta):\*\* Administra el bucle de eventos del servidor web HTTP integrado (basado en `libmicrohttpd`). Despacha las peticiones HTTP entrantes, invoca el enrutador REST de forma asíncrona y gestiona los sockets abiertos.
2. `svr.Discovery`` (Prioridad Media):\*\* Implementa el descubrimiento automático de cámaras IP mediante peticiones de red usando protocolos basados en ONVIF y WS4D (Web Services for Devices).
3. `svr.NTP`` y `svr.NTPState`` (Prioridad Baja):\*\* Se encargan de la sincronización de reloj del sistema. Levantan peticiones hacia servidores de tiempo externos y aplican correcciones de retardo en red.
4. `Telnet Server`` / `TelnetSlave`` (Prioridad Baja):\*\* Exponen una interfaz de consola interactiva clásica para la administración remota a través de comandos estructurados.
5. `svr.proxy.master`` (Prioridad Alta):\*\* Realiza la conmutación y enrutamiento en tiempo real de los flujos de streaming de vídeo de cámaras remotas hacia el cliente web.
6. `svr.IfTop`` y `svr.Top`` (Prioridad Baja):\*\* Monitorizan las estadísticas de uso de CPU, ocupación de sockets TCP e interfaces de red físicas para generar reportes dinámicos.
7. `drv.Interphone`` (Prioridad Media):\*\* Controla el procesamiento del canal de audio bidireccional y la comunicación de intercomunicadores IP.

---

### ## 3. Inicio de Sesión (Login) y Criptografía Híbrida

La seguridad en el inicio de sesión evita que las contraseñas viajen legibles en texto plano o que puedan interceptarse y replicarse mediante ataques de repetición.

#### #### El Protocolo de Intercambio de Clave RSA-AES

##### 1. \*\*Generación de Claves RSA Temporales:\*\*

Al iniciar la aplicación, el cliente (navegador con Angular 19) genera un par de claves asimétricas RSA de 512 o 1024 bits de forma local utilizando la API Web Crypto.

##### 2. \*\*Negociación del Secreto Simétrico:\*\*

El cliente realiza un POST al endpoint `/rpc/GetEncryptedAES16Key`` enviando la parte pública de su clave RSA: el módulo `PublicKeyN`` y el exponente `PublicKeyE`` en formato de texto hexadecimal.

##### 3. \*\*Creación del Secreto AES-128 y la Sesión:\*\*

- \* `la-core-legacy`` intercepta la petición, verifica si hay una cookie de sesión en la cabecera del cliente (`cgi_get_http_cookie_session_id``). Si no existe, invoca a `web_login_prepare_new_session`` para inicializar una sesión en memoria y asignarle un identificador único.

- \* El servidor genera una **clave simétrica AES-128 aleatoria** específica para esa sesión.

- \* Cifra la clave AES utilizando la clave pública RSA del cliente (`newRSA``) y el algoritmo de relleno `RSA_PKCS1_PADDING``.

- \* Codifica la clave cifrada en formato hexadecimal y la devuelve en la respuesta JSON: `{"GetEncryptedAES16KeyResponse": "<hex_string>"}`.

- \* El cliente recibe la respuesta y la cookie de sesión, y descifra la clave AES simétrica con su clave privada RSA.

#### 4. \*\*Autenticación Cifrada Simétrica:\*\*

- \* El cliente cifra los datos del usuario (`User` y `Password`) con la clave simétrica AES-128 en modo CBC.

- \* Codifica el resultado en Base64 y lo envía al endpoint `/rpc/Login` en el parámetro `EncryptedPassword`.

- \* El backend recupera la clave AES asociada a la sesión de la cookie, decodifica el Base64, extrae el IV de los primeros 16 bytes y descifra el texto con OpenSSL o la librería criptográfica embebida (`AesCbcDecrypt`).

- \* Las credenciales en claro resultantes son validadas a través de `command\_user\_verify\_all`.

#### ### Prevención de Ataques de Fuerza Bruta y Replay

El sistema de seguridad contra ataques de repetición y fuerza bruta está regulado por `replay\_attack.cpp`:

- \* Si el login falla, se almacena el número de intentos fallidos por dirección IP física llamando a `replay\_attack\_process\_unauthorized\_auth`.

- \* Si el número de reintentos supera el umbral parametrizado en la directiva del grabador, la dirección IP se marca en estado `locked` (bloqueado).

- \* Cualquier petición posterior de login de esa IP es rechazada de inmediato sin verificar las credenciales, respondiendo: `{"LoginResponse":"locked"}`.

- \* En caso de fallo por debajo del umbral, se retorna el código `unauthorized` y la cantidad de intentos restantes: `{"LoginResponse":"unauthorized","RetryCount":"X"}`.

---

#### ## 4. Modelo de Seguridad y Roles (Grants)

Una vez autenticado el usuario, el sistema le asocia una máscara de bits que define sus privilegios (Grants). La verificación se realiza de forma estricta en el enrutador REST antes de despachar cada endpoint HTTP.

#### ### Tabla de Privilegios del Grabador (Grants Map)

| Privilegio / Grant           | Máscara Hex | Descripción Operativa                                                                                                          |
|------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|
| :---                         | :---        | :---                                                                                                                           |
| `USER_PERMISO_GESTION`       | `0x001`     | Operaciones críticas: reinicio, reset de fábrica, importación de firmware, formateo y visualización de cargas de CPU globales. |
| `USER_PERMISO_CONFIGURACION` | `0x002`     | Lectura, parcheado y commit de propiedades generales de hardware y software (red, cámaras, canales de audio).                  |
| `USER_PERMISO_VIVO`          | `0x004`     | Acceso a flujos de vídeo en directo. Autoriza la inicialización de proxies de vídeo.                                           |
| `USER_PERMISO_GRABACIONES`   | `0x008`     | Acceso para realizar búsquedas e iniciar reproducción de vídeo almacenado en el disco.                                         |
| `USER_PERMISO_DESCARGAS`     | `0x010`     | Descarga de ficheros y exportación de streams grabados en formato propietario o estándar.                                      |

| `USER\_PERMISO\_ASISTENTE` | `0x020` | Ejecución del asistente interactivo básico de instalación inicial. |  
 | `USER\_PERMISO\_ASISTENTE\_PLUS` | `0x040` | Ejecución de asistentes avanzados y parametrizaciones guiadas de red e IP. |  
 | `USER\_PERMISO\_UPDATE\_FW` | `0x080` | Capacidad para invocar la instalación de parches apk y binarios de firmware. |  
 | `USER\_PERMISO\_VIVO\_LIMITADO` | `0x100` | Acceso restringido a ciertas cámaras en directo. |  
 | `USER\_PERMISO\_CONFIG\_USERS` | `0x200` | Creación, eliminación y mapeo de perfiles de usuario. Acceso exclusivo a `/superuser/password`. |  
 | `USER\_PERMISO\_FTP` | `0x400` | Acceso a recursos de servidor FTP local del grabador. |

### ### Validación de Privilegios en el Enrutador

El enrutador REST (`api\_rest\_dispatch.cpp`) valida los permisos correspondientes mediante la estructura `Entity\_ApiRestElementNodeTree`:

1. Obtiene el grant de la sesión HTTP mediante la llamada `cgi\_get\_grant(sock)`.
2. Extrae los privilegios necesarios según el nodo solicitado y el tipo de método HTTP. Por ejemplo, una consulta GET de propiedades puede requerir un nivel inferior a una actualización PUT o un parche PATCH.
3. Compara mediante una operación a nivel de bits: `(grants & permiso\_requerido)`.
4. Si la operación resulta en cero, interrumpe el procesamiento y retorna una respuesta JSON de error estándar (`g\_json\_str\_api\_rest\_service\_error\_privilege`) junto con un código de estado HTTP `403 Forbidden`.

---

## ## 5. Jerarquía REST y Catálogo Detallado de Endpoints

El servicio declara estáticamente los endpoints en `api\_rest\_globals.cpp`. Cada endpoint está representado por un nodo de árbol de tipo `Entity\_ApiRestElementNodeTree`.

A continuación se muestra el catálogo de endpoints del sistema con su respectivo método HTTP y la máscara de privilegios asociada:

### ### Grupo A: Servicio de Dispositivos (Device Service)

| Ruta Relativa                       | Método | Callback Asociado                               | Permiso Requerido            |
|-------------------------------------|--------|-------------------------------------------------|------------------------------|
| :/device-service/license`           | GET    | `api_rest_device_service_license_get`           | `USER_PERMISO_TODOS`         |
| :/device-service/license`           | PUT    | `api_rest_device_service_license_put`           | `USER_PERMISO_GESTION`       |
| :/device-service/properties/schema` | GET    | `api_rest_device_service_properties_schema_get` | `USER_PERMISO_CONFIGURACION` |
| :/device-service/properties/data`   | GET    | `api_rest_device_service_properties_data_get`   | `USER_PERMISO_CONFIGURACION` |



```

| `/device-service/properties/data` | POST | `api_rest_device_service_properties_data_post`
| `USER_PERMISO_CONFIGURACION` |
| `/device-service/properties/data` | PUT | `api_rest_device_service_properties_data_put` |
| `USER_PERMISO_CONFIGURACION` |
`/device-service/properties/data`	PATCH
`api_rest_device_service_properties_data_patch`	`USER_PERMISO_CONFIGURACION`
`/device-service/properties/data`	DELETE
`api_rest_device_service_properties_data_delete`	`USER_PERMISO_CONFIGURACION`
`/device-service/properties/lock`	GET
`USER_PERMISO_CONFIGURACION`	
`/device-service/properties/unlock`	GET
`api_rest_device_service_properties_unlock_get`	`USER_PERMISO_CONFIGURACION`
`/device-service/properties/locklist`	GET
`api_rest_device_service_properties_locklist_get`	`USER_PERMISO_CONFIGURACION`
`/device-service/configuration`	GET
`USER_PERMISO_GESTION`	
`/device-service/factory-state/configuration`	PUT
`api_rest_device_service_factory_state_configuration_put`	`USER_PERMISO_GESTION`
`/device-service/factory-state/restore`	PUT
`api_rest_device_service_factory_state_restore_put`	`USER_PERMISO_GESTION`
`/device-service/errors`	GET
`USER_PERMISO_TODOS`	
`/device-service/errors/{id}`	PUT
`USER_PERMISO_CONFIGURACION`	
`/device-service/superuser/password`	PUT
`api_rest_device_service_superuser_password_put`	`USER_PERMISO_CONFIG_USERS`

```

### ### Grupo B: Servicio de Recursos (Resources Service)

```

Ruta Relativa	Método	Callback Asociado	Permiso Requerido
`/device-service/resources/users`	GET	`api_rest_device_service_resource_users_get`	
`USER_PERMISO_TODOS`			
`/device-service/resources/users/map`	GET		
`api_rest_device_service_resource_users_map_get`	`USER_PERMISO_TODOS`		
`/device-service/resources/users/map/profiles`	GET		
`api_rest_device_service_resource_users_map_profiles_get`	`USER_PERMISO_TODOS`		
`/device-service/resources/clock`	GET	`api_rest_device_service_resource_clock_get`	
`USER_PERMISO_TODOS`			
`/device-service/resources/clock/timezones`	GET		
`api_rest_device_service_resource_clock_timezones_get`	`USER_PERMISO_TODOS`		
`/device-service/resources/clock/datetime`	GET/PUT		
`api_rest_device_service_resource_clock_datetime_get/put`	`USER_PERMISO_TODOS`		
(GET) / `USER_PERMISO_CONFIGURACION` (PUT) |

```

```
| `/device-service/resources/video/cameras` | GET |
`api_rest_device_service_resource_video_cameras_get` | `USER_PERMISO_TODOS` |
| `/device-service/resources/video/ipcameras` | GET |
`api_rest_device_service_resource_video_ipcameras_get` | `USER_PERMISO_TODOS` |
| `/device-service/resources/video/ipcameras/lock` | GET/PUT |
`api_rest_device_service_resource_video_ipcameras_lock_get/put` |
`USER_PERMISO_CONFIGURACION` |
| `/device-service/resources/hd` | GET | `api_rest_device_service_resource_hd_get` |
`USER_PERMISO_TODOS` |
| `/device-service/resources/hd/format` | PUT |
`api_rest_device_service_resource_hd_format_put` |
`USER_PERMISO_CONFIGURACION` |
| `/device-service/resources/io-space/outputs/commute` | GET |
`api_rest_device_service_resource_io_space_outputs_commute_get` |
`USER_PERMISO_TODOS` |
| `/device-service/proxy/rules` | GET/POST | `api_rest_device_service_proxy_rules_get/post`
| `USER_PERMISO_TODOS` (GET) / `USER_PERMISO_CONFIGURACION` (POST) |
| `/device-service/proxy/rules/{id}` | PUT/DELETE |
`api_rest_device_service_proxy_rules_id_put/delete` |
`USER_PERMISO_CONFIGURACION` |
```

### ### Grupo C: Servicio de Diagnósticos e Informes (Reports Service)

```
Ruta Relativa	Método	Callback Asociado	Permiso Requerido
`/device-service/reports/io-space`	GET	`api_rest_device_service_reports_io_get`	
`USER_PERMISO_TODOS`			
`/device-service/reports/reset/core_dumps`	GET		
`api_rest_device_service_reports_reset_core_dumps_get`	`USER_PERMISO_TODOS`		
`/device-service/reports/reset/reset_list`	GET		
`api_rest_device_service_reports_reset_reset_list_get`	`USER_PERMISO_TODOS`		
`/device-service/reports/system_load/cpu`	GET		
`api_rest_device_service_reports_system_load_cpu_get`	`USER_PERMISO_TODOS`		
`/device-service/reports/system_load/network`	GET		
`api_rest_device_service_reports_system_load_network_get`	`USER_PERMISO_TODOS`		
`/device-service/reports/video/stats`	GET		
`api_rest_device_service_reports_video_stats_get`	`USER_PERMISO_TODOS`		
`/device-service/reports/video/detail`	GET		
`api_rest_device_service_reports_video_detail_get`	`USER_PERMISO_TODOS`		
`/device-service/reports/video/image`	GET		
`api_rest_device_service_reports_video_image_get` | `USER_PERMISO_TODOS` |
```

---

### ## 6. Gestión de Configuración y Bloqueo Pesimista (Pessimistic Locking)

Para garantizar la integridad y evitar condiciones de carrera, el grabador utiliza un mecanismo de **bloqueo pesimista jerárquico** en las propiedades del sistema.

#### ### Funcionamiento del Lock/Unlock y Tokens

##### 1. **Solicitud de Lock:**

El frontend envía una petición a

`/device-service/properties/lock?node=config.cameras.001&seed=987654321``.

##### 2. **Verificación de Colisiones y Adquisición:**

- \* El servicio adquiere un mutex de exclusión mutua global para proteger la tabla en memoria `g_active_locks``.

- \* Valida si el nodo exacto está bloqueado.

- \* **Validación Jerárquica:** Comprueba si existe algún bloqueo sobre un nodo antecesor (por ejemplo, si un administrador ha bloqueado la raíz `config``, no se permite bloquear `config.cameras.001``). Igualmente, si existe un bloqueo en un nodo hijo (por ejemplo, `config.cameras.001.enabled``), se deniega el bloqueo sobre `config.cameras.001``.

- \* Si el recurso está disponible, se genera una estructura de bloqueo en el backend, asignando un `token`` único y aleatorio de 32 caracteres y un tiempo de expiración (por defecto 30 segundos).

##### 3. **Refresco del Temporizador:**

El cliente debe enviar una petición periódica antes de que transcurran los 30 segundos.

Si no se recibe actividad, el hilo de limpieza elimina el bloqueo de la estructura del servidor.

##### 4. **Desbloqueo Explícito:**

Se realiza enviando el token al endpoint `/unlock``.

#### ### Modificación Parcial con JSON Patch (RFC 6902)

El endpoint `/device-service/properties/data`` procesa operaciones PATCH complejas cuando la cabecera HTTP `Content-Type`` contiene el valor `application/json-patch+json``.

##### \* **Aplanamiento de Estructuras (Flat Map):**

El formateador traduce la estructura JSON del parche en un mapa clave-valor plano.

##### \* **Soporte de Wildcards (`{}``):**

Si la ruta de parche contiene comodines (por ejemplo, `config.cameras.{}.enabled``), la función busca recursivamente los identificadores de canales válidos cargados en la base de datos (por ejemplo, `001``, `002``) y expande el parche replicando la operación para cada canal.

##### \* **Filtros Condicionales (`where``):**

El parche acepta la expresión `where`` para filtrar la expansión de comodines basándose en las propiedades actuales (por ejemplo, `"where": "ip_camera == true"` aplicará el parche solo sobre canales que tengan configuradas cámaras IP).

#### ### Validación Estructural y Esquemas JSON

Antes de almacenar el cambio en la base de datos, el backend lee el esquema JSON de propiedades invocando externamente a `sys_prop_validator`` (`remote_validate_list``). Este motor valida:

1. El tipo de datos del valor modificado (entero, booleano, string, etc.).
2. Los límites de rango permitidos (por ejemplo, máscara de red válida, rango de puertos entre 1 y 65535, longitudes de texto).
3. Compatibilidad del esquema actual.

### ### Control de Reinicio Requerido (`check\_if\_reboot\_needed\_smart`)

El grabador evalúa si la modificación introducida requiere reiniciar el hardware para poder aplicarse:

- \* Se escanean las propiedades modificadas en la base de datos.
- \* Se localiza la definición de cada una de estas propiedades en el esquema JSON (`find\_property\_schema`).
- \* Se inspecciona si contiene el atributo `\_rebootRequired': true`.
- \* **Comprobación Inteligente (Smart Check):** Se compara el valor actual en la base de datos con el valor propuesto en la petición de escritura. Si los valores son idénticos, no se marca como reinicio pendiente. Si difieren y la propiedad está catalogada como crítica, se invoca `system\_mark\_reboot\_required`).
- \* El servidor marca la bandera global `global\_reboot\_required = true` y añade en la cabecera HTTP de la respuesta de escritura el flag `X-Reboot-Required: true`.

---

## ## 7. El Motor de Asistentes de Configuración Progresiva (PCFG y Wizard2)

Para la puesta en marcha rápida del NVR, el sistema incluye asistentes secuenciales basados en una máquina de estados controlada por el backend.

### ### El Módulo Progressive Configuration (PCFG)

El módulo `pcfg` (`root/webinteface/pcfg/`) implementa un asistente de configuración progresivo que permite definir los parámetros iniciales del grabador manteniendo los datos en una estructura temporal aislada antes de aplicarlos.

#### 1. **Activación de Sesión de Asistente:**

Se realiza accediendo al recurso virtual `pcfg\_init.html` que activa `wizard2.cgi\_enter\_pcfg`. Esto inicializa el contexto de almacenamiento intermedio `pcfgVariables`.

#### 2. **Máquina de Estados de Pasos (Steps Flow):**

El asistente progresa a través de una enumeración secuencial de pasos. A continuación se desglosa el flujo principal:

...

[Inicio: pcfg\_init.html]

|

v

[Paso 1: Configuración General del Grabador] ---> (pcfg\_general\_step\_0.cpp)

|

v

[Paso 2: Creación de Usuarios y Claves] ---> (pcfg\_general\_step\_users.cpp)

|

v

[Paso 3: Ajustes de Reloj y Zona Horaria] ---> (pcfg\_clock\_process.cpp)

|

v

[Paso 4: Parametrización de Red e interfaces] -> (pcfg\_hd\_step\_4.cpp)

```

 |
 v
[Paso 5: Configuración de Cámaras IP] ----> (wz2_ipCam_step.cpp)
 |
 v
[Paso 6: Definición de Almacenamiento y RAID] -> (pcfg_hd_save_variables.cpp)
 |
 v
[Paso 7: Calendarios y Reglas de Eventos] ----> (pcfg_step_event_horarios.cpp)
 |
 v
[Paso Final: Validación y Commit Atómico] ----> (pcfg_save_changes.cpp)
...

```

\* **\*\*Vídeo e IP:\*\*** Durante el paso de cámaras IP, el backend ejecuta ``wz2_ipCam_step.cpp`` para evaluar si las cámaras descubiertas son aptas. Acepta llamadas dinámicas como ``PCFG_VIDEO_STEP_IP_SETTINGS`` y ``PCFG_VIDEO_STEP_IP_PRIVACY_MASK`` para ajustar las máscaras de privacidad del flujo de vídeo.

\* **\*\*Horarios y Eventos:\*\*** Controla la creación de franjas de grabación horarias (``pcfg_crear_elemento_horario.cpp``) asociadas a acciones de entrada y alarma (``pcfg_crear_elemento_evento.cpp``).

\* **\*\*Guardado y Commit:\*\*** Cada paso escribe sobre ``pcfgVariables`` llamando a su función de guardado respectiva (ej. ``pcfg_general_save_variables``). En el paso final, ``pcfg_save_changes`` realiza una validación cruzada y escribe todas las propiedades de una sola vez hacia la base de datos central a través de D-Bus, minimizando los reinicios del sistema.

#### ### El Módulo Wizard2

El sistema ``wizard2`` (``root/core/wizard2/``) expone los asistentes tradicionales interactivos:

- \* Mapea los endpoints ``wizard2_index.html``, ``wizard2_init.html`` y ``wizard2_main.html`` bajo los privilegios ``USER_PERMISO_ASISTENTE`` y ``USER_PERMISO_ASISTENTE_PLUS``.
- \* Provee recursos de interfaz interactiva, como el control deslizante de almacenamiento (``wizard2_slider.js``), validaciones de formularios en el cliente (``wizard2_validation.js``) y previzualizaciones HTML del flujo RTSP de las cámaras (``wizard2_video.js``).

---

#### ## 8. Pipeline de Hardware, RAID, Telemetría y Watchdog del Sistema

##### ### Telemetría y Estadísticas del Motor de Vídeo

A través del endpoint ``/device-service/reports/video/stats``, el backend consulta asíncronamente las métricas de rendimiento del decodificador multimedia embebido. Invocando ``video_engine_call_get_receivers_report()``, el servicio recupera un mapa en memoria con los estados de los receptores de red:

- \* ``receiver``: Índice de la cámara emisora.
- \* ``bw``: Ancho de banda detectado en kilobits por segundo (kbps).
- \* ``rx_pkts``: Paquetes recibidos.

- \* ``jitter``: Fluctuación del retardo temporal en red (milisegundos).
- \* ``lost``: Número absoluto de paquetes de vídeo perdidos detectados por la cabecera RTP.

### ### Niveles RAID y Gestión de Almacenamiento

``la-core-legacy`` permite supervisar y configurar el subsistema de almacenamiento del NVR, soportando:

1. `**Single (Código `99`)**` Disco único sin tolerancia a fallos.
2. `**Aggregate (Código `0`)**` Concatena discos para sumar capacidad sin paridad (RAID 0).
3. `**Mirror (Código `1`)**` Redundancia en espejo simple entre dos unidades físicas.
4. `**Double Mirror (Código `2`)**` Doble conjunto en espejo para configuraciones de 4 discos.
5. `**RAID Dinámico (Código `3`)**` Un modo de red propietario (RAID LAN) que replica la información del grabador en caliente hacia otro grabador Lanaccess a través de sockets TCP seguros en red local.

La llamada REST ``/hd/format`` evalúa el modo RAID configurado en ``hdConf.IRaidMode`` y ejecuta ``hd_erase_one_disk_format`` (para discos independientes o espejos individuales) o ``hd_erase_all_disks_format`` (para configuraciones dinámicas y agregados).

### ### El Sistema de Watchdog y Causas de Reset

El grabador monitoriza de forma constante la salud física y de software. En caso de anomalías críticas, el servicio fuerza un reinicio controlado para recuperar el funcionamiento.

La función ``wr_reboot`` registra la causa (``ucCause``), el usuario (``strUsername``), guarda el log en la memoria flash no volátil (``wr_persistent_reset_list_add``), duerme 3 segundos y establece flags de eventos en la estructura global ``flReboot`` (``WR_HARDWARE_REBOOT`` o ``WR_SOFTWARE_REBOOT``) para que el hilo supervisor ``wr_thread_reboot`` ejecute la secuencia del sistema (ya sea enviando una señal a la fuente de alimentación física mediante GPIO o ejecutando el comando de apagado seguro de Linux ``popen("reboot")``).

A continuación se detalla la lista de códigos de diagnóstico de reinicio (``wr_reset_string.cpp``):

| Constante de Causa                  | Explicación Técnica del Fallo                                                                    |
|-------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>`R_H_REBOOT`</code>           | Reinicio solicitado de forma manual por un administrador a través del menú de configuración.     |
| <code>`R_SOFTWARE_CHANGED`</code>   | Reinicio necesario tras aplicar cambios mayores en propiedades del sistema de bajo nivel.        |
| <code>`R_INCORRECT_FIRMWARE`</code> | Firmware no compatible detectado en arranque.                                                    |
| <code>`R_H_POWER_UP`</code>         | Encendido inicial de la fuente de alimentación.                                                  |
| <code>`R_H_WATCHDOG`</code>         | El watchdog de hardware de la placa no ha recibido el "ping" a tiempo y ha cortado la corriente. |
| <code>`R_ME_SOFTWARE_REBOOT`</code> | Hilos principales del software colgados o caídos detectados por el supervisor en Linux.          |

| `R\_TCPIP\_MEMORY\_ERR` | Fuga de memoria o desbordamiento en la pila de red TCP/IP embebida. |

| `R\_NO\_SOCKET\_ERR` | Agotamiento absoluto de sockets de sistema disponibles en red. |

| `R\_HARD\_DISK\_ERR\_X\_R` | Fallo de lectura física irrecuperable en el disco duro número X. |

| `R\_HARD\_DISK\_ERR\_X\_W` | Fallo de escritura física irrecuperable en el disco duro número X. |

| `R\_HARD\_DISK\_FATAL\_ERR\_X` | El disco X reporta un fallo de estado SMART crítico que imposibilita la grabación. |

| `R\_HARD\_DISK\_TIMEOUT\_X` | Latencia excesiva de respuesta en operaciones I/O en la unidad SATA X. |

| `R\_WATCHDOG\_SOFTWARE` | Watchdog por software de ThreadX. Un hilo de alta prioridad ha monopolizado la CPU por más de 10 segundos. |

| `R\_CFG\_BAD\_DATA\_MEMORY` | Datos corruptos en la base de datos de propiedades al cargar la flash en arranque. |

| `R\_SWITCH\_ERR` | Error de comunicación con el circuito integrado de Switch de red local integrado. |

| `R\_CONFIG\_UPDATE` | Se ha aplicado un paquete completo de actualización de base de datos. |

---

## ## 9. Guía de Integración Práctica con el Frontend Angular 19

Para interactuar con el backend expuesto por `la-core-legacy`, el frontend desarrollado en Angular 19 (`web-core-view`) debe implementar los siguientes patrones de comunicación y consumo de APIs:

### ### 1. Interceptor de Sesión y Autenticación Criptográfica

Se requiere desarrollar un guard y un interceptor que implementen la criptografía híbrida.

```
``typescript
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent } from
'@angular/common/http';
import { Observable, from } from 'rxjs';
import { mergeMap } from 'rxjs/operators';

@Injectable()
export class HybridCryptographyInterceptor implements HttpInterceptor {
 private aesKey: CryptoKey | null = null;

 intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
 if (req.url.endsWith('/rpc/Login')) {
 // Cifrar credenciales antes de enviar
 return from(this.encryptPayload(req.body)).pipe(
 mergeMap(encryptedBody => {
```

```

 const secureReq = req.clone({ body: encryptedBody });
 return next.handle(secureReq);
 })
);
}
return next.handle(req);
}

private async encryptPayload(body: any): Promise<any> {
 // 1. Convertir JSON a array de bytes
 const textEncoder = new TextEncoder();
 const data = textEncoder.encode(JSON.stringify({ User: body.username, Password:
body.password }));

 // 2. Generar vector de inicialización (IV) de 16 bytes
 const iv = window.crypto.getRandomValues(new Uint8Array(16));

 // 3. Cifrar con la clave AES negociada mediante AES-CBC
 const encryptedData = await window.crypto.subtle.encrypt(
 { name: 'AES-CBC', iv: iv },
 this.aesKey!,
 data
);

 // 4. Concatenar IV + Datos Cifrados
 const combined = new Uint8Array(iv.length + encryptedData.byteLength);
 combined.set(iv);
 combined.set(new Uint8Array(encryptedData), iv.length);

 // 5. Retornar JSON para el RPC
 return {
 User: body.username,
 EncryptedPassword: this.arrayBufferToBase64(combined.buffer)
 };
}

private arrayBufferToBase64(buffer: ArrayBuffer): string {
 let binary = "";
 const bytes = new Uint8Array(buffer);
 for (let i = 0; i < bytes.byteLength; i++) {
 binary += String.fromCharCode(bytes[i]);
 }
 return window.btoa(binary);
}
}
...

```

## ### 2. Gestión del Ciclo de Vida del Bloqueo (Locks) en Formularios Angular



Para evitar sobreescrituras en la edición de propiedades, se implementa un control reactivo en el componente del formulario:

```
``typescript
import { Component, OnInit, OnDestroy } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Subject, interval } from 'rxjs';
import { takeUntil, switchMap } from 'rxjs/operators';

@Component({
 selector: 'app-config-form',
 templateUrl: './config-form.component.html'
})
export class ConfigFormComponent implements OnInit, OnDestroy {
 private destroy$ = new Subject<void>();
 private lockToken: string | null = null;
 private nodePath = 'config.network';

 constructor(private http: HttpClient) {}

 ngOnInit() {
 this.acquireLock();
 }

 acquireLock() {
 const seed = Math.floor(Math.random() * 1000000000).toString();
 this.http.get<{ token: string }>(`/device-service/properties/lock?node=${this.nodePath}&seed=${seed}`)
 .subscribe(res => {
 this.lockToken = res.token;
 this.startLockRenewal();
 });
 }

 startLockRenewal() {
 // Renovamos el lock cada 20 segundos (antes de que expire a los 30 segundos)
 interval(20000).pipe(
 takeUntil(this.destroy$),
 switchMap(() => {
 const seed = Math.floor(Math.random() * 1000000000).toString();
 return
this.http.get(`/device-service/properties/lock?node=${this.nodePath}&seed=${seed}&token=
${this.lockToken}`);
 })
).subscribe();
 }

 releaseLock() {
```

```

 if (this.lockToken) {

this.http.get(`/device-service/properties/unlock?node=${this.nodePath}&token=${this.lockToken}`)
 .subscribe(() => {
 this.lockToken = null;
 });
 }
}

ngOnDestroy() {
 this.destroy$.next();
 this.destroy$.complete();
 this.releaseLock();
}
}
...

```

### ### 3. Interceptor para Detección de Reinicios ('X-Reboot-Required')

Un servicio global intercepta las respuestas HTTP para detectar si se ha guardado una propiedad que requiera reiniciar el NVR:

```

``typescript
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler, HttpEvent, HttpResponse } from
'@angular/common/http';
import { Observable } from 'rxjs';
import { tap } from 'rxjs/operators';
import { MatSnackBar } from '@angular/material/snack-bar';

@Injectable()
export class RebootRequiredInterceptor implements HttpInterceptor {
 constructor(private snackBar: MatSnackBar) {}

 intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
 return next.handle(req).pipe(
 tap(event => {
 if (event instanceof HttpResponse) {
 const rebootHeader = event.headers.get('X-Reboot-Required');
 if (rebootHeader === 'true') {
 this.snackBar.open(
 'Configuración crítica modificada. Es necesario reiniciar el grabador para aplicar los
cambios.',
 'Reiniciar ahora',
 { duration: 10000 }
).onAction().subscribe(() => {
 this.triggerSystemReboot();
 });
 }
 }
 })
);
 }
}

```

```
 }
 }
 })
);
}

private triggerSystemReboot() {
 // Envía una petición PUT al endpoint de reset para apagar y encender el grabador
 // (Requiere USER_PERMISO_GESTION)
}
}
...
```